

Геопространственная оценка малых районов*

Практическое руководство

Хаои Чэнь[†]

Джошуа Д. Мерфельд[‡]

Дэвид Ньюхаус[§]

Ричард Пирс Тонкин[¶]

Оглавление

1	Введение	3
1.1	Оценка малых районов с помощью геопространственных данных	3
1.2	О данном руководстве	4
1.3	Структура руководства	5
2	Установка языка программирования R	7
2.1	RStudio	7
2.2	Пакеты	8
3	Файлы векторных данных	10
3.1	Чтение и визуализация файлов форм	11
3.2	Система отсчета координат	13
3.3	Где мы можем найти файлы форм?	16
4	Растры	17
4.1	Чтение и визуализация растров	18
4.2	Извлечение растровых данных в файлы форм	19
4.3	Создание файла формы из точек	24
4.4	Создание сетки	29

* Данное руководство было разработано при поддержке проекта ЭСКАТО «Большие данные для официальной статистики», финансируемого Субфондом по осуществлению Повестки дня в области устойчивого развития на период до 2030 года Целевого фонда мира и развития ООН. В данном руководстве встречаются карты для визуализации как геопространственных данных, так и результатов их использования. Указанные на картах границы, названия и обозначения не означают официального одобрения или признания со стороны Организации Объединенных Наций.

[†] СОООН

[‡] Квинслендский университет и IZA

[§] Всемирный банк и IZA

[¶] ЭСКАТО

4.5	Где мы можем найти растры?	32
4.6	Worldpop	32
4.7	Mosaiks	33
4.8	Использование языка Python для доступа к сервису Google Earth Engine.....	34
4.9	Проект Geolink.....	42
4.10	Заключительный этап.....	42
5	Данные обследования	43
5.1	Подготовка данных обследования.....	43
6	Создание и выбор объектов	45
6.1	Создание новых объектов, например, административных средних.....	45
6.2	Размышления о преобразованиях	47
7	Оценка модели	58
7.1	Пакет rounpar.....	58
7.2	Определение таких опций, как взвешивание, преобразование, задание эталонных значений.....	59
7.3	Проверка предположений.....	60
7.4	Оценка результатов	62
8	Представление бедности в виде карты распределения	67
9	Подведение итогов	69
	Библиографический указатель	70
	Приложение	71

1 Введение

Оценка малых районов (ОМР) – это набор статистических методов, позволяющих статистикам получать оценки интересующих их результатов на уровнях агрегирования, когда объем выборки слишком мал для получения надежных прямых оценок. Под «малыми районами», как правило, подразумеваются административные или другие географические территории. Вместе с тем, к ним могут относиться и подсовокупности, для которых объем выборки слишком мал для надежной оценки параметров.

ОМР активно развивается уже несколько десятилетий ([Ghosh 2020](#)), и по этой теме написано несколько книг (особенно хорошей отправной точкой является книга Rao and Molina ([2015](#))). При всем многообразии реализаций ОМР, все методы имеют схожую концепцию – во всех случаях основная идея заключается в том, чтобы «дополнить» данные опроса вспомогательными данными, которые могут предсказать результат.

Более подробно о различных методах ОМР и их применении в официальной статистике можно узнать на веб-странице электронной энциклопедии проекта SAE4SDGs [здесь](#).

1.1 Оценка малых районов с помощью геопространственных данных

Традиционно ОМР опирается на данные переписи населения на уровне отдельных единиц или на высококачественные административные данные, охватывающие все рассматриваемое население, а не только обследуемые территории. При наличии таких данных можно с высокой степенью вероятности прогнозировать многие интересующие нас результаты, включая показатели бедности, безработицы и другие показатели ЦУР. Однако ключевая проблема, с которой сталкиваются многие страны, заключается в том, что такие данные либо недоступны, либо устарели. В связи с этим возникает важный вопрос: если у нас нет доступа к административным данным, то какие данные мы можем использовать, которые:

1. Могут предсказать интересующий результат?
2. Доступны для всех слоев населения во всех районах, представляющих интерес?

Геопространственные данные, получаемые со спутниковых снимков и других систем наблюдения за Землей, являются одним из таких источников данных, отвечающих перечисленным требованиям. Преимущества геопространственных данных для целей ОМР заключаются в следующем:

- Глобальная доступность, в том числе в регионах с дефицитом данных;
- Высокая частота обновления – как правило, ежегодно или чаще;
- Прогностическая способность для уровня бедности и связанных с ним результатов (напр., индекс NDVI, ночная освещенность, плотность населения).

При этом использование геопространственных данных сопряжено с определенными трудностями. Прежде всего, использование геопространственных данных непривычно для многих политиков и сотрудников национальных статистических организаций. Другими словами, использование геопространственных данных может потребовать

относительно быстрого обучения, особенно когда речь идет о доступе к данным через API и использовании альтернативных языков программирования (главным образом R и Python). Для этого был разработан [начальный учебник по оценке малых территорий с использованием геопространственных данных](#), который содержит общий обзор и практические рекомендации по использованию геопространственных данных для оценки малых районов (OMR). В нем рассматриваются наиболее часто используемые геопространственные переменные и общедоступные хранилища данных этого типа.

1.2 О данном руководстве

Перед вами пошаговое руководство по использованию геопространственных данных для оценки малых районов с помощью языка программирования R.

Руководство призвано дополнить вышеупомянутый учебник и посвящено практическим аспектам реализации. Для облегчения практического обучения в руководство включены фрагменты кода. Читателям предлагается скопировать и запустить код на своем компьютере, следуя руководству. Для этого рекомендуется использовать HTML-версию данного руководства, доступную в репозитории GitHub. Там также содержатся все необходимые данные.

В этом руководстве мы будем использовать данные по Северной провинции Малави. Источником данных обследования является [Пятое комплексное обследование домохозяйств \(IHS5\)](#), которое считается репрезентативным лишь на уровне района (admin2). Нашей конечной целью будет оценка уровня бедности для Северной провинции Малави на уровне admin3, что невозможно при использовании первичных данных обследования – для многих районов уровня admin3 вообще нет наблюдений, а те, что есть, как правило, имеют небольшой размер выборки.

Для этого у нас есть три варианта. Во-первых, мы можем оценить модель на уровне домохозяйств, соединив данные обследования с геопространственными данными. Во-вторых, мы можем оценить модель на уровне admin4 (в случае Малави – на уровне счетного участка (СУ)), агрегируя данные обследования до уровня admin4 и подключая геопространственные данные на том же уровне. Последний вариант – создать сетку, охватывающую всю Северную провинцию Малави, и агрегировать все данные до уровня сетки, чтобы оценить уровни бедности. Мы надеемся, что к концу этого руководства у вас будут инструменты для проведения оценки по любому из трех вариантов.

На практике выбор варианта зачастую зависит от имеющихся у вас файлов форм (.shp). Например, если у вас нет геокоординат домохозяйства (долгота и широта по GPS), но есть admin-идентификатор, который можно использовать для отождествления с файлом форм, вы не сможете оценить модель на уровне сетки, но вы сможете оценить модель на уровне домохозяйства, а затем агрегировать оценки на более высоком уровне. Аналогично, если у вас нет файлов форм на уровне admin4, вы не сможете оценить модель на этом уровне, но сможете оценить модель на уровне домохозяйства или сетки.

Хотя в руководстве используются примеры данных по Малави, методику и код можно адаптировать к другим странам и условиям. Примеры предполагают базовое понимание языка R, но код полностью аннотирован, чтобы пользователи могли вносить изменения и опираться на него.

Из соображений краткости мы не будем подробно останавливаться на самом языке R, за исключением тех случаев, когда это применимо к конкретным задачам. Другими словами, мы предполагаем, что читатели имеют базовое представление о языке R и его синтаксисе. Хотя мы будем приводить весь код, который используем, мы не всегда будем разъяснять детали синтаксиса. Также в данном руководстве не будет подробно рассмотрена теория оценки малых районов – за более подробной информацией рекомендуем обратиться к электронной энциклопедии проекта SAE4SDGs и другим ресурсам, упомянутым выше.

1.3 Структура руководства

В первых разделах этого руководства рассматриваются основные пакеты для языка R, которые мы будем использовать, а также различные типы данных, необходимых для геопространственной ОМР. В разделе 2 мы обсудим настройку языка R и пакетов, которые будем использовать, включая интегрированную среду разработки (IDE), которую мы будем использовать (RStudio, раздел 2.1), а также все пакеты, которые мы будем использовать в этом руководстве (раздел 2.2).

Затем мы рассмотрим различные форматы данных: файлы форм (векторные данные) и файлы растровых данных. Геопространственные данные обычно хранятся в форматах, которые некоторые пользователи, возможно, раньше не видели, а тем более не использовали. Поэтому мы уделим значительное время обсуждению этих форматов данных и изучению того, как работать с ними в языке R. В разделе 3 мы обсудим, как читать и создавать файлы векторных данных, которые в данном контексте мы обычно называем файлами форм, а также где их можно найти. Мы будем использовать пакет языка R под названием `terra` для работы как с файлами форм, так и с файлами растровых данных. В разделе 4 мы обсудим, как читать и создавать файлы растровых данных, а также как извлекать растровые данные в файлы форм. Мы также обсудим, как создавать сетки, если файл формы недоступен, но у нас есть GPS-координаты в данных обследования.

К данным обследования предъявляются особые требования, чтобы обследование можно было увязать с геопространственными данными. В разделе 5 мы рассмотрим, как выполнить такое увязывание, а также шаги по подготовке обследования.

Затем мы перейдем к обсуждению того, какие предикторы использовать в ОМР-модели. Наиболее распространенные на сегодняшний день модели ОМР, как правило, являются линейными моделями, а это значит, что нам нужно быть осторожными с включением слишком большого количества предикторов, как для того, чтобы предотвратить чрезмерную подгонку (предсказание «помех» в данных), так и для того, чтобы элементарно иметь возможность оценить модель (если предикторов больше, чем наблюдений, оценка будет невозможна). Поэтому в разделе 6 мы обсудим, как выбирать объекты, создавать новые объекты и продумывать преобразования. Мы также обсудим, как использовать пакет `glmnet` для отбора объектов методом лассо-регрессии.

После этого мы перейдем к оценке моделей ОМР. В разделе 7 мы обсудим, как оценить модель с помощью пакета `poymap`, а также как задать опции, проверить предположения и оценить результаты.

Наконец, в разделе [8](#) мы обсудим, как составить карту бедности, используя полученные оценки. Мы создадим таблицы и карты для визуализации результатов.

Весь использованный код и данные вы можете найти в репозитории GitHub для этого руководства [здесь](#).

2 Установка языка программирования R

Прежде чем приступить к работе с этим руководством, на вашем компьютере уже должен быть установлен язык R. Если еще нет, вы можете скачать его [здесь](#).

Перед началом работы *настоятельно* рекомендуется создать на своем компьютере новую папку, в которой будут храниться скрипты (раздел 2.1), все загруженные вами данные и все созданные вами выходные данные. Это поможет все упорядочить и облегчит поиск файлов в будущем. Это также облегчит работу над данным руководством, поскольку вы сможете легко остановиться и продолжить позже.

Если вы хотите в точности следовать шагам, описанным в этом руководстве, вам также следует создать подпапку с названием `data` в своей новой папке. В папке `data` вы будете хранить все загруженные с GitHub данные. Однако в качестве рабочей директории всегда должна использоваться основная папка, а не папка `data`.

2.1 RStudio

Основная цель заключается в использовании языка программирования R для оценки малых районов с помощью геопространственных данных. Для этого мы будем использовать RStudio – бесплатную интегрированную среду разработки с открытым исходным кодом для языка R. Хотя использование среды RStudio не является обязательным, она обладает множеством функций, облегчающих работу с языком R, особенно для тех, кто только изучает этот язык. Вы можете скачать RStudio [здесь](#)¹.

Ключевой особенностью IDE является возможность запуска кода из скрипта. Скрипт, по сути, является текстовым файлом, содержащим только тот код, который вы хотите запустить. Скрипты особенно полезны для воспроизводимости, так как позволяют сохранить весь написанный код и запускать его повторно в любое время. В среде RStudio вы можете создать новый скрипт, последовательно щелкнув по пунктам меню `File > New File > R Script`.

После написания кода вы можете запустить его, щелкнув по кнопке `Run` в правом верхнем углу окна скрипта или нажав соответствующую комбинацию клавиш². Вы можете запустить одну строку кода, поместив курсор на эту строку и нажав комбинацию клавиш. Кроме того, вы можете выделить несколько строк кода и запустить их все аналогичным образом. Важное замечание: если вы хотите выделить несколько строк кода для запуска, вы *должны* выделить каждую строку целиком; если вы выделите только часть строки, код не будет запущен.

Важным дополнением к созданию новых папок и нового скрипта является указание этой папки в качестве рабочего каталога. Это можно сделать несколькими способами:

¹ Существует множество альтернативных IDE. Например, [Visual Studio Code](#) имеет расширение, которое можно использовать для написания и выполнения кода на языке R. В числе других IDE – такие, как [Jupyter](#) и [Positron](#), последняя из которых все еще находится в стадии разработки. Однако в данном руководстве мы не будем рассматривать ни одну из этих альтернатив.

² На ПК под управлением Windows по умолчанию используется комбинация клавиш `Ctrl + Enter`, на Mac – комбинация клавиш `Cmd + Enter`.

1. Если RStudio *не запущен*, открытие скрипта из папки автоматически задаст эту папку (расположение скрипта) в качестве рабочего каталога.
2. Если RStudio *запущен*, вы можете задать рабочий каталог, последовательно щелкнув по пунктам меню Session > Set Working Directory > To Source File Location. Это задаст расположение скрипта в качестве рабочего каталога.
3. Вы также можете задать рабочий каталог с помощью функции `setwd()`. Например, если на вашем рабочем столе есть папка `geospatialSAE`, вы можете задать ее в качестве рабочего каталога, прописав `setwd("~/Desktop/geospatialSAE")` или что-то аналогичное, в зависимости от файловой структуры вашей машины. В целом же, мы рекомендуем не делать этого в вашем скрипте. Причина проста: если вы поделитесь своим скриптом с кем-то еще, то рабочий каталог для них будет не таким, как для вас. Вместо этого мы предлагаем использовать первые два метода, которые являются более воспроизводимыми.

2.2 Пакеты

В этом руководстве мы будем использовать следующие пакеты:

- tidyverse
- terra
- tidyterra
- povmap
- glmnet
- haven

Чтобы начать работу, вам нужно установить эти пакеты. Это можно сделать, выполнив следующий код:

```
1 install.packages(c("tidyverse", "terra", "tidyterra", "povmap", "glmnet", "haven"))
```

На любом конкретном компьютере пакеты нужно устанавливать только один раз. Поэтому не обязательно включать их в скрипт; вместо этого можно установить их из консоли.

При этом, несмотря на то, что устанавливать пакеты нужно лишь единожды, загружать их придется каждый раз, когда вы начинаете новый сеанс работы с языком R. Это делается с помощью функции `library()` следующим образом³:

³ Обратите внимание на использование хештега (`#`) для создания комментария в строке 1. Это заставляет язык R игнорировать строку, и она не будет выполнена. Такие комментарии используются для того, чтобы предоставить читателю пояснения.

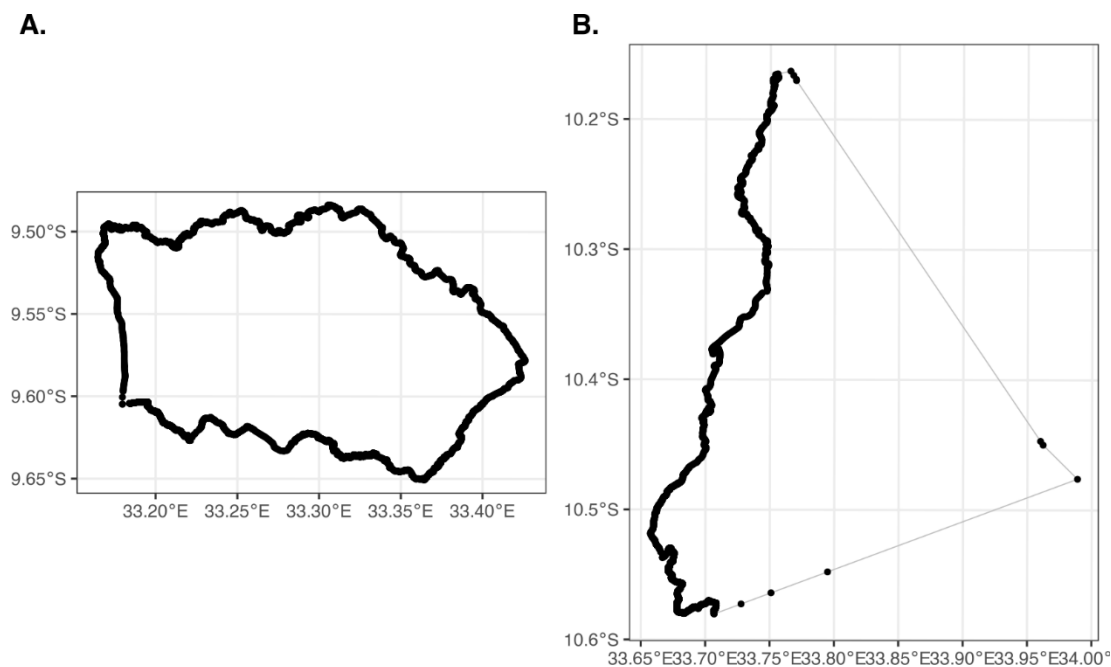
```
1 # Загрузка библиотек
2 library(tidyverse)
3 library(terra)
4 library(tidyterra)
5 library(povmap)
6 library(glmnet)
7 library(haven)
```

По сложившейся практике скрипты на языке R всегда следует начинать с загрузки библиотек, которые вы будете использовать. Так вы можете быть уверены, что все нужные вам функции будут доступны.

3 Файлы векторных данных

Файлы векторных данных являются одним из наиболее распространенных форматов геопространственных данных, и в геопространственном контексте их принято называть «файлами форм»⁴. С ними знакомы практически все и даже те, кто не работал с ними напрямую: многие карты, которые вы видите, состоят из файлов форм! Файлы форм – это контуры географических областей или других представляющих интерес объектов. Например, в файле форм могут быть очертания зданий, хотя чаще всего это контуры административных районов. В файлах форм контуры этих «полигонов» очерчиваются точками. Рассмотрим простой квадрат: вы можете очертить весь квадрат, используя всего четыре точки. Для пятиугольника потребуется пять точек, для шестиугольника – шесть, и так далее. Файлы форм – это просто список точек, или вершин, которые очерчивают контуры различных объектов/полигонов. Размер файла формы увеличивается не из-за географической площади объекта как такового (с точки зрения объема памяти компьютера), а из-за количества вершин, необходимых для очерчивания формы. Некоторые географические границы имеют относительно гладкие края, что приводит к уменьшению количества вершин, в то время как другие, такие как береговые линии, представляют собой сложные формы, требующие гораздо большего количества вершин. Файлы форм могут содержать описательную информацию о каждом объекте – например, некоторые страны выпускают файлы форм, включающие демографическую информацию, полученную в результате переписи населения, но именно географическая природа файлов форм делает их файлами форм.

Рисунок 1: Вершины объектов файла формы



⁴ Файлы векторных данных существуют не только в геопространственном контексте. Например, векторные данные содержатся как в файлах .svg, так и в файлах .pdf.

Рассмотрим два примера на рис. 1. Обе фигуры получены из файла формы территории традиционных властей (ТВ) Малави; ТВ считаются третьим уровнем административного деления (admin3) страны. Площадь территории на рис. А (слева) составляет всего 345,8 кв. км, в то время как площадь территории на рис. В (справа) – 777,9 кв. км. Несмотря на то, что площадь территории на рис. В более чем в два раза больше, рис. А имеет 6280 вершин, а рис. В – 2583 вершины. Это объясняется тем, что рис. А – относительно более сложная фигура, а рис. В – более простая. Если бы мы сохранили эти две отдельные территории ТВ как файлы форм, то файл с рис. А был бы намного больше, чем файл с рис. В, несмотря на меньшую географическую площадь.

Ключевой особенностью файлов форм является то, что они зачастую состоят из множества отдельных объектов. В случае с рис. 1 слева и справа показаны два отдельных объекта из одного файла формы. В данном случае эти два объекта называются полигонами, поскольку они представляют собой замкнутые объекты с множеством сторон. Однако файлы форм могут также включать точки (напр., местоположение домохозяйства), линии (напр., дорога) или даже множественные полигоны (напр., несколько островов, входящих в состав одной ТВ). В этом руководстве мы будем работать в основном с полигонами и точками.

И последнее замечание: «файл формы» – это не совсем верное название. Мы имеем в виду файл с расширением `.shp`, но на самом деле файл формы – это набор файлов. Файл с расширением `.shp` содержит географические данные, но к нему обычно прилагаются другие файлы, содержащие различную информацию. В целом, для правильного чтения файла формы *необходимо* иметь как минимум три файла: файл `.shp`, файл `.shx` и файл `.dbf`. Еще один файл, который не является строго необходимым, но нередко включается в комплект, – это файл `.prj`, который содержит информацию о проекции файла формы. Если у вас нет файла `.prj`, вам может потребоваться указать проекцию вручную при чтении файла формы. Если вы хотите узнать больше об этих различных файлах и содержащейся в них информации, в Интернете можно найти множество кратких объяснений, в том числе на веб-сайтах геоинформационных систем [ESRI](#) и [QGIS](#).

3.1 Чтение и визуализация файлов форм

Для чтения файлов форм в языке R мы будем использовать пакет `terra`. В папке данных в репозитории GitHub находится файл формы под названием `mw3.shp`. Вы заметите, что там есть еще несколько файлов с такими же именами, но с разными расширениями. Этот файл формы содержит контуры территорий традиционных властей (ТВ) Северной провинции Малави. Чтобы прочитать этот файл формы, мы можем использовать следующий код (предварительно загрузив библиотеку с помощью команды `library(terra)` в верхней части скрипта, как обсуждалось в разделе 2.2):

```
1 # Загрузка файла формы admin3 для Малави mw3
2 <- vect("data/mw3.shp")
3 # посмотрим на результат
4 mw3
```

```

class      : SpatVector
geometry   : polygons
dimensions : 76, 2 (geometries, attributes)
extent     : 493675.9, 691460, 8591761, 8964834 (xmin, xmax, ymin, ymax)
source     : mw3.shp
coord. ref.: Arc 1950 / UTM zone 36S
names      : TA_CODE      DIST_CODE
type       :      <chr>      <chr>
values     :      10120      101
              10110      101
              10102      101

```

Этот код также загружает объект `mw3` в строке 4, выводя информацию о файле формы в консоль. Продвигаясь вниз по строкам, мы видим следующую информацию:

- `class: SpatVector`: Это просто означает, что мы загрузили файл формы с помощью пакета `terra`.
- `geometry: polygons`: В файл формы входят только полигоны; в нем нет точек и линий.
- `dimensions: 76, 2 (geometries, attributes)`: В файле формы 76 отдельных объектов и два атрибута (или переменные/столбца). В нижней части вывода мы можем видеть названия двух атрибутов (`TA_CODE` и `DIST_CODE`), тип переменных (обе они – символьные (т.е. `<chr>`), а не числовые) и несколько примеров значений.
- `extent: 493675.9, 691460, 8591761, 8964834 (xmin, xmax, ymin, ymax)`: Охват файла формы – это координаты рамки, которая полностью окружает весь файл формы (т.н. «ограничительная рамка»). На рис. [A1](#) в приложении показана ограничительная рамка синего цвета ООН для файла формы `mw3.shp`. Заметим, что координаты в охвате не похожи на те, которые мы обычно считаем координатами, то есть они указаны не в широте и долготе! Вместо этого они указаны в метрах. Мы вернемся к этому ниже, в разделе [3.2](#).
- `source: mw3.shp`: Имя загруженного нами файла формы (в том виде, в котором он сохранен на нашем компьютере).
- `coord. ref.: Arc 1950 / UTM zone 36S`: Это система отсчета координат (CRS). Мы обсудим ее более подробно в разделе [3.2](#).

Для визуализации файла формы мы будем использовать `ggplot2` из пакета `tidyverse`⁵ и функцию `geom_spatvector()` из пакета `tidyterra`.

```

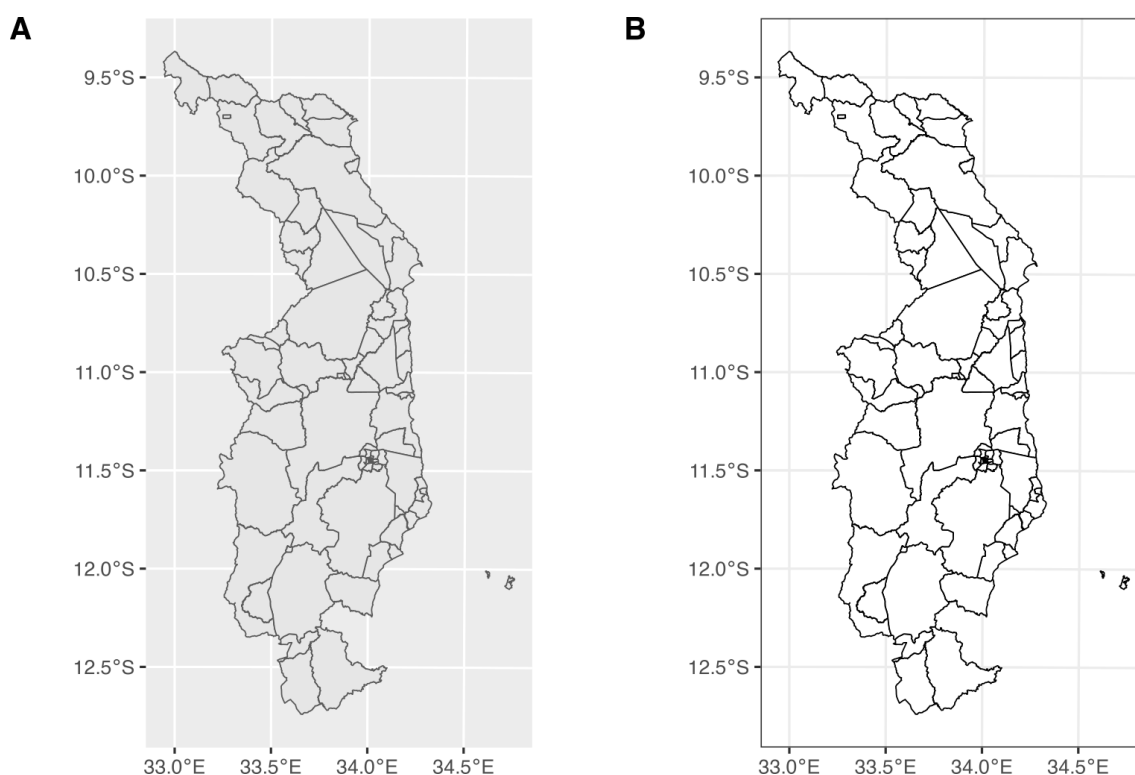
1 # Рисунок А
2 ggplot() +
3   geom_spatvector(data = mw3)
4 # Рисунок В
5 ggplot() +
6   geom_spatvector(data = mw3, color = "black", fill = "white") +
7   theme_bw(base_size = 8)

```

⁵ Когда вы загружаете библиотеку `tidyverse`, она автоматически подгружает несколько других пакетов, включая `ggplot2`. Это означает, что вам не нужно загружать пакет `ggplot2` отдельно.

Приведенный выше код выдает два отдельных примера с незначительными различиями в коде для каждого из них. На рис. А рисунка 2 показана тема по умолчанию без каких-либо изменений. Получается серый фон, серая «заливка» объектов и черный «контур» объектов. На рис. В показаны небольшие изменения, сделанные из эстетических соображений⁶. Мы изменили заливку на белую (white), то есть внутренние цвета всех объектов будут отображаться белыми. Кроме того, мы использовали одну из тем, встроенных в ggplot2 – это тема `theme_bw()`, которая меняет фон карты⁷.

Рисунок 2: Традиционные власти (admin3) в Северной провинции Малави



3.2 Система отсчета координат

Основная сложность отображения карт на двумерной поверхности заключается в том, что Земля – это шар⁸. «Проецирование» трехмерного объекта на двумерную поверхность по своей природе является сложной задачей. Основная трудность заключается в том, что при создании двумерного изображения невозможно идеально сохранить как форму, так и площадь объекта. Именно поэтому иногда можно видеть карты земного шара, которые несколько отличаются друг от друга⁹.

⁶ Это совершенно необязательно и является вопросом личных предпочтений.

⁷ Полный список встроенных тем можно найти [здесь](#).

⁸ Земля на самом деле представляет собой *эллипсоид вращения* (так как она выпуклая на экваторе), но для наших целей достаточно рассматривать ее как сферу.

⁹ На [веб-сайте QGIS](#) есть несколько отличных примеров этого в разделе 8.4.

Наиболее распространенной системой координат, особенно в повседневной жизни, являются долгота и широта. Однако технически это не является проекцией, поскольку относится к местоположению точки на сфере Земли. Проекция – это преобразование, которые переносят трехмерную форму Земли на плоскую двумерную плоскость. Существует множество различных проекций, но одна из самых распространенных – универсальная поперечная проекция Меркатора (UTM). Эта проекция делит Землю на 60 отдельных «зон», каждая из которых охватывает шесть градусов долготы. Не вдаваясь в подробности, можно сказать, что принципиальная разница между координатами UTM и GPS заключается в том, что координаты UTM указываются в метрах, а координаты GPS – в градусах. Именно поэтому охват файла формы `mw3.shp` указан в метрах, а не в градусах. Возвращаясь к приведенному выше результату, мы видим, что зона CRS – это «зона UTM 36S»: именно в этой зоне находится Малави, страна, показанная в нашем файле формы.

Для небольших стран, таких как Малави, вы получите очень похожие результаты вне зависимости от того, используете ли вы UTM или долготу/широту при расчете площади и/или расстояний. Тем не менее, некоторые различия все же будут наблюдаться.

Сначала сконвертируем координаты файла формы в координаты долготы/широты. Для этого нам нужен код «EPSG» для соответствующей системы CRS¹⁰. Код EPSG для наших долготы/широты – 4326. Мы можем использовать функцию `project()` из пакета `terra` для конвертации координат файла формы в координаты долготы/широты. Затем мы найдем площадь и длину периметра, используя как проекцию UTM, так и проекцию координат долготы/широты¹¹:

```
1 mw3_latlon <- project(mw3, "EPSG:4326")
2 # найдем площадь:
3 mw3_latlon$area <- expanse(mw3_latlon)
4 mw3$area <- expanse(mw3, transform = FALSE)
5 # найдем длину периметра:
6 mw3_latlon$perimeter <- perim(mw3_latlon)
7 mw3$perimeter <- perim(mw3)
8 summary(mw3)
```

TA_CODE	DIST_CODE	area	perimeter
Length :76	Length :76	Min. :8.414e+05	Min. : 4314
Class :character	Class :character	1st Qu. :2.148e+07	1st Qu. : 30625
Mode :character	Mode :character	Median :1.668e+08	Median : 75867
		Mean :3.570e+08	Mean : 91986
		3rd Qu. :4.870e+08	3rd Qu. : 130190
		Max. :1.969e+09	Max. : 324527

```
1 summary(mw3_latlon)
```

¹⁰ Аббревиатура «EPSG» расшифровывается как European Petroleum Survey Group (Европейская группа нефтяных исследований). В настоящее время этого названия уже нет, но аббревиатура продолжает использоваться.

¹¹ По умолчанию функции `expanse` и `perim` возвращают значения в квадратных метрах, а не километрах.

TA_CODE	DIST_CODE	area	perimeter
Length :76	Length :76	Min. :8.418e+05	Min. : 4315
Class :character	Class :character	1st Qu. :2.149e+07	1st Qu. : 30632
Mode :character	Mode :character	Median :1.669e+08	Median : 75884
		Mean :3.572e+08	Mean : 92015
		3rd Qu. :4.873e+08	3rd Qu. : 130228
		Max. :1.970e+09	Max. : 324324

Насколько велики различия? Для площади наибольшая абсолютная разница составляет 0,09 процента. Для периметра наибольшая абсолютная разница составляет 0,13 процента. Хотя для Малави эти различия невелики, при расчете для более обширных территорий на Земле они могут оказаться весьма значительными. Пакет `terra` прямо предлагает использовать долготу/широту для этих расчетов; собственно, именно поэтому мы должны указать `transform = FALSE` в функции `expand()` выше. По умолчанию для вычисления площади пакет `terra` преобразует любой файл формы в долготу/широту¹².

Однако важнее убедиться, что все используемые вами пространственные объекты имеют координаты в *одной и той же* CRS. В то время как некоторые функции конвертируют их автоматически, другие этого не делают. В последнем случае иногда будет выдана ошибка (напр., «They are not in the same CRS. Please project» (Объекты в разных системах CRS. Пожалуйста, измените систему отсчета координат), а иногда функция сработает, но не вернет ожидаемых результатов. Поэтому важно всегда проверять систему CRS ваших пространственных объектов и ввести в привычку преобразование всех пространственных объектов в одну и ту же систему CRS перед проведением пространственного анализа.

Для наших текущих целей вполне достаточно координат долготы/широты. Мы уже видели, как явно изменить проекцию с помощью кода EPSG. Однако мы можем сделать это и с помощью другого объекта. Теперь у нас есть два отдельных объекта «mw3»: `mw3` и `mw3_latlon`. Давайте сконвертируем объект `mw3` в ту же систему CRS, что и объект `mw3_latlon` (который имеет координаты долготы/широты):

```
1 mw3 <- project(mw3, crs(mw3_latlon))
2 # проверяем
3 mw3
```

```
class      : SpatVector
geometry    : polygons
dimensions  : 76, 4 (geometries, attributes)
extent      : 32.94264, 34.7591, -12.74079, -9.36724 (xmin, xmax, ymin, ymax)
coord. ref. : lon/lat WGS 84 (EPSG:4326)
names       : TA_CODE  DIST_CODE      area  perimeter
type        : <chr>    <chr>         <num>   <num>
values      : 10120     101      8.504e+06  1.243e+04
              10110     101      7.775e+08  1.468e+05
              10102     101      3.455e+08  1.022e+05
```

¹² Кроме того, в справочном файле функции `perim(?terra::perim)` прямо говорится: «Если система отсчета координат не является долготой/широтой, можно получить более точные результаты, предварительно преобразовав данные в долготу/широту с помощью функции `project`».

Как вы можете видеть, параметр `coord. ref` для объекта `mw3` теперь имеет значение `lon/lat WGS 84 (EPSG:4326)`, что указывает на то, что теперь он имеет координаты долготы/широты. Кроме того, вы заметите, что изменился охват: теперь он указан в GPS-координатах, а не в метрах.

3.3 Где мы можем найти файлы форм?

Так где же можно найти файлы форм? К сожалению, единого ответа нет. Во многих странах национальные агентства геопространственной информации (НАГИ) или геопространственные или географические подразделения других агентств, связанных с данными, могут создавать и распространять официальные файлы форм. Например, на веб-сайте Корейского национального географического института можно найти файлы форм для Кореи (вплоть до уровня `admin4`). Однако в некоторых странах такие файлы форм не публикуются.

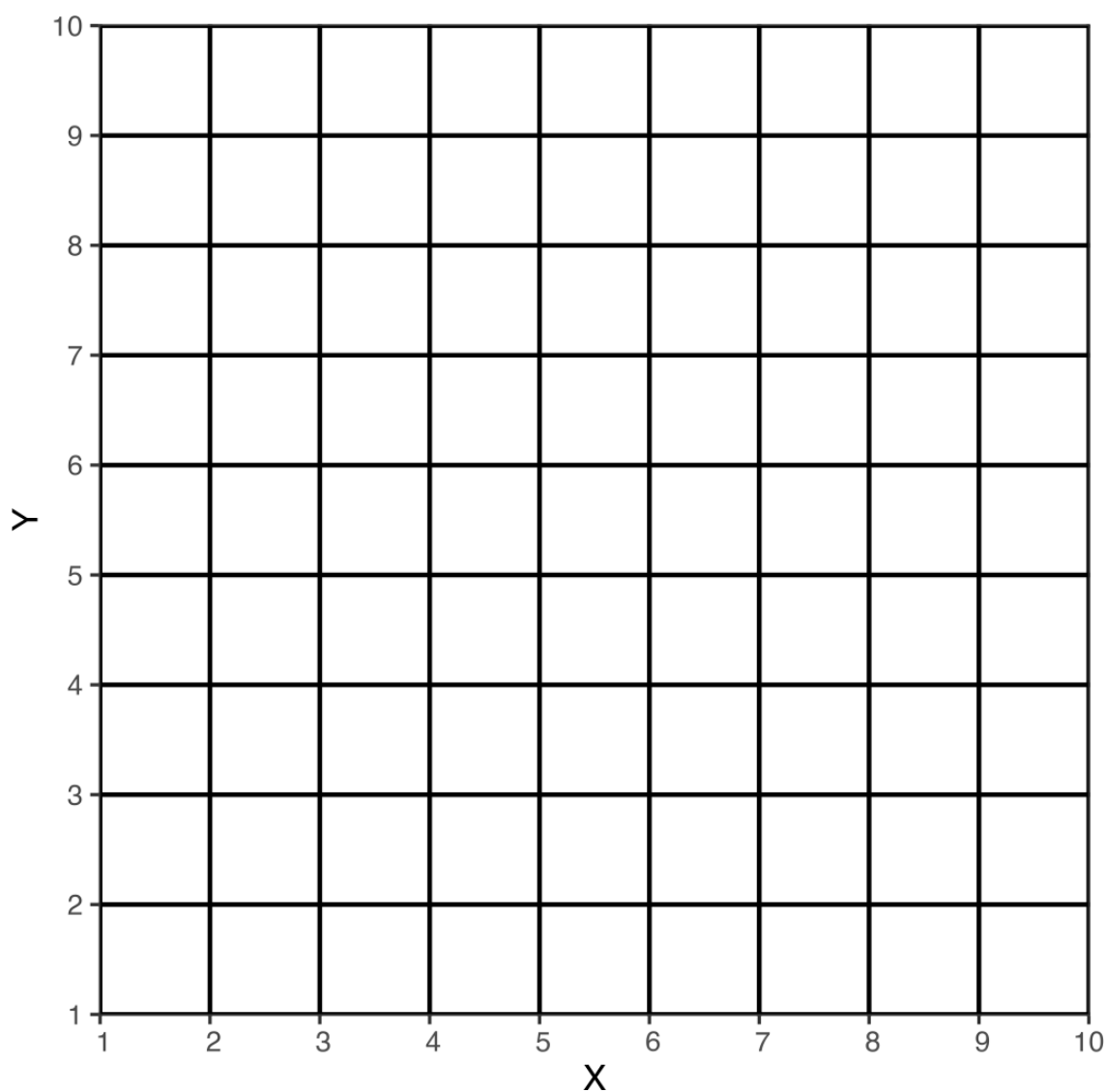
Другим надежным источником является ресурс по обмену гуманитарными данными¹³ Управления ООН по координации гуманитарных вопросов (УКГВ). При необходимости файлы форм можно также найти с помощью простого поиска в Интернете. Однако следует с осторожностью относиться к источнику и происхождению таких файлов, особенно если предполагается использовать их в официальных целях.

¹³ <https://data.humdata.org/>

4 Растры

Обычно мы используем файлы форм для оконтуривания административных районов или географических объектов различных типов. Однако файлы форм редко используются для хранения данных. Причина проста: объем памяти. Файлы форм просто не являются эффективными для хранения больших объемов данных, по крайней мере, по сравнению с растровыми данными. Тем не менее, использование файлов форм для *извлечения* данных с целью оценки моделей ОМР является обычной практикой. Например, в последующих разделах мы будем оценивать модель ОМР на уровне СУ (admin4) в Малави. Это означает, что нам необходимо агрегировать все предикторы до уровня СУ, что мы и сделаем, используя файл формы и растровые данные уровня СУ.

Рисунок 3: Пример растровых данных



Растры – это другой тип геопространственных данных. Вместо того чтобы очерчивать полигоны точками, растры состоят из сетки, каждая ячейка которой содержит значение (или значения) – как, например, сетка на рис. 3. Если бы рис. 3 был файлом формы, для каждой отдельной ячейки потребовалось бы пять вершин¹⁴. Но растр – это совсем другое. Поскольку каждая ячейка сетки имеет одинаковые размеры, нам нужно знать только две вещи, чтобы определить местоположение *всех* ячеек сетки в пространстве: одну точку в углу (или центроид) сетки и размеры ячеек сетки! Это делает хранение данных в растрах гораздо более удобным, чем хранение данных в файлах форм. Компромисс, конечно же, заключается в том, что все ячейки имеют одинаковую форму, в то время как объекты файла формы могут быть любой формы.

Мы будем иметь дело с растрами в контексте геопространственных данных, но растры просто обозначают формат, в котором они хранятся. Многие распространенные форматы файлов изображений также являются растрами. Например, файлы .png и .jpg являются растрами – изображения состоят из множества отдельных ячеек сетки, пусть и без геопространственного компонента, который мы будем использовать.

4.1 Чтение и визуализация растров

Как и в случае с файлами форм, мы будем читать растры с помощью пакета `terra`. В папке `data` в репозитории GitHub вы найдете файл растровых данных с названием `ndviexample.tif`. Этот файл содержит индекс растительности NDVI за апрель 2019 года. Чтобы прочитать его, мы можем использовать следующий код:

```
1 ndvi <- rast("data/ndviexample.tif")
2 # проверяем
3 ndvi

class      : SpatRaster
dimensions : 377, 203, 1 (nrow, ncol, nlyr)
resolution : 0.008983153, 0.008983153 (x, y)
extent      : 32.94122, 34.7648, -12.74709, -9.360445 (xmin, xmax, ymin, ymax)
coord. ref. : lon/lat WGS 84 (EPSG:4326)
source      : ndviexample.tif
name        : NDVI
```

Как и раньше, мы можем вывести данные объекта (`ndvi`) в консоль и просмотреть некоторые итоговые данные. Большая часть информации похожа на ту, что мы видели в файле формы, но есть и некоторые различия.

- `class: SpatRaster`: Это просто означает, что мы загрузили растр с помощью пакета `terra`.
- `dimensions: 377, 203, 1 (nrow, ncol, nlyr)`: Растр состоит из 377 строк, 203 столбцов и одного слоя. Под «слоем» понимается количество каналов (или

¹⁴ Хотя у квадрата всего четыре угла, пятая точка необходима для того, чтобы *замкнуть* фигуру и создать полигон. Другими словами, первая и последняя точки находятся в одной точке.

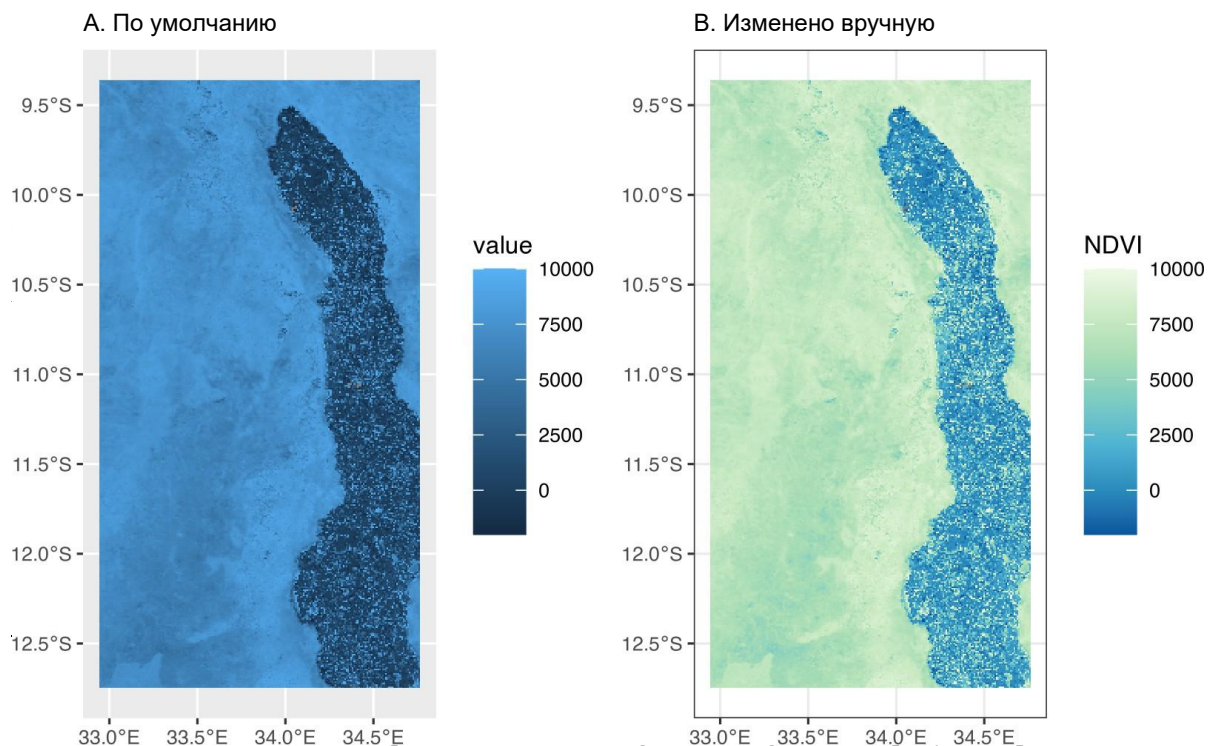
переменных) в растре. В этом примере растр содержит только одну единицу информации: NDVI.

- resolution: 0.008983153, 0.008983153 (x, y): Разрешение – это размер каждой ячейки сетки. В данном случае CRS – это долгота/широта, то есть разрешение в *градусах*, а не в метрах.
- name: NDVI: Названия – это названия слоев/каналов/переменных. Опять же, в этом растре есть только одна переменная, и ее название – NDVI.

Для визуализации растра мы воспользуемся функцией `geom_spatraster` из пакета `ggplot`. Эта функция автоматически построит растровое изображение с цветовой шкалой, которая идет от минимального до максимального значения в растре. В данном случае наш растр имеет только одну переменную, поэтому нам не нужно пока беспокоиться о других особенностях. Мы можем визуализировать растр с помощью следующего кода:

```
1 # Рисунок А
2 ggplot() +
3   geom_spatraster(data = ndvi)
4 # Рисунок В
5 ggplot() +
6   geom_spatraster(data = ndvi) +
7   scale_fill_distiller(palette = "GnBu") +
8   theme_bw()
```

Рисунок 4: Растровая визуализация индекса NDVI для Северной провинции Малави



Стоит на минутку остановиться и вспомнить, к чему мы стремимся в итоге. Наша конечная цель – оценить модель малого района с помощью геопространственных данных. Для этого нам нужно *извлечь* растровые данные в файл формы; этот файл формы может содержать полигоны уровня admin4 (СУ в случае Малави), полигоны сетки или точки домохозяйств. Другими словами, нам нужны предикторы из растров (напр., индекс NDVI или уровень ночной освещенности), агрегированные до уровня admin4. В этом разделе мы покажем, как извлечь растровые данные в файл формы.

Если мы хотим оценить модель на уровне admin4, мы извлечем растровые данные в файл формы уровня admin4. Этот процесс «извлечения» наложит растр на файл формы и найдет различные «плитки» растра, которые перекрывают каждый полигон в файле формы, чтобы объединить их с помощью некоей выбранной функции¹⁷. Мы можем сделать это с помощью функции `extract()` из пакета `terra`. Функция `extract()` берет растровые данные и извлекает их в файл формы. Сначала загрузим файл формы уровня admin4 в среду R, используя функцию `vect` из пакета `terra`¹⁸:

```
1 # загрузим admin4
2 mw4 <- vect("data/mw4.shp")
3 # посмотрим результат
4 mw4
```



```
class       : SpatVector
geometry    : polygons
dimensions  : 3212, 3 (geometries, attributes)
extent      : 32.94242, 34.75888, -12.74058, -9.367346 (xmin, xmax, ymin, ymax)
source      : mw4.shp
coord. ref. : lon/lat WGS 84 (EPSG:4326)
names       : DIST_CODE      EA_CODE      TA_CODE
type        : <chr>          <chr>        <chr>
values      :      105        10507801    10507
              105          10507072    10507
              105          10507010    10507
```

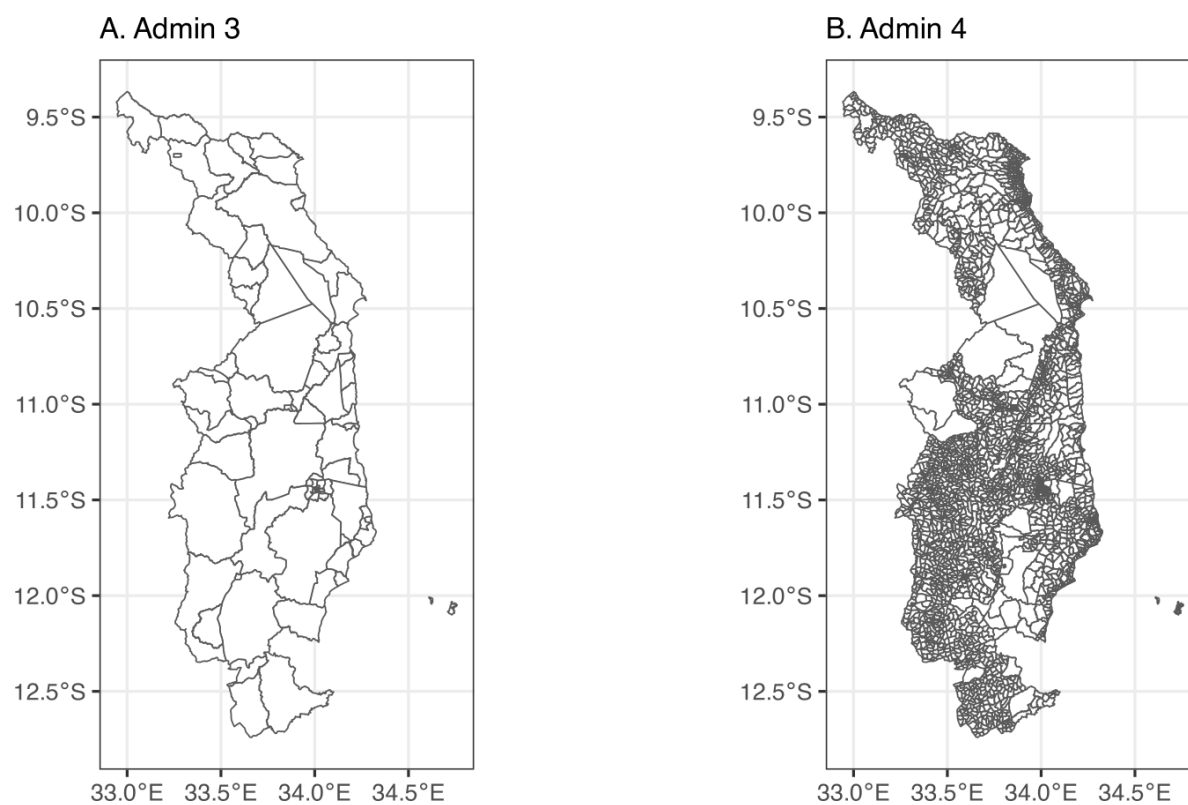
Здесь мы видим тот же результат, что и при просмотре файла формы уровня admin3, но с другими значениями, поскольку файл формы уровня admin4 имеет другие атрибуты (и больше объектов). Мы можем увидеть разницу наглядно на рис. 5.

На рисунке слева показан контур областей уровня admin3, а на рисунке справа – контур областей уровня admin4. Области admin4 меньше по размеру и более многочисленны, чем области admin3 (3212 объектов против 76 объектов). Хотя они покрывают одну и ту же географическую область, файл формы уровня admin3 занимает 4,9 МБ памяти, в то время как файл формы admin4 занимает 40,3 МБ. Мы упомянули об этом, потому что большие файлы форм могут иногда приводить к проблемам с памятью на некоторых компьютерах, особенно при извлечении данных из больших растров.

¹⁷ К примеру, для оценки ночной освещенности мы можем прибегнуть к агрегированию, взяв среднее значение из растровых плиток, которые накладываются на данный полигон. Для оценки населения, с другой стороны, мы, вероятно, захотим взять их сумму.

¹⁸ Файл `adm4.shp` можно найти в папке `data`.

Рисунок 5: Сравнение уровней административных единиц в Северной провинции Малави



Мы хотим извлечь данные индекса NDVI в файл формы `admin4` таким образом, чтобы каждый объект (географическая область) в файле формы `admin4` имел среднее значение NDVI для этого объекта. Мы можем сделать это с помощью функции `extract()` из пакета `terra`, которая создаст блок данных со средним значением NDVI для каждого объекта. Код выглядит следующим образом:

```
1 # обратите внимание на порядок и на то, что мы хотим получить СРЕДНЕЕ ЗНАЧЕНИЕ (mean):
2 extractedndvi <- extract(ndvi, mw4, fun = "mean")
3 head(extractedndvi)
```

	ID	NDVI
1	1	6225.000
2	2	6624.000
3	3	6112.167
4	4	6883.500
5	5	6684.200
6	6	6252.667

Функция `extract()` принимает три аргумента: растровые данные, файл формы и функцию для применения. В данном случае мы хотим получить среднее значение индекса NDVI для каждого объекта, поэтому используем `fun = "mean"`. На выходе мы получаем блок данных с двумя столбцами, но важно то, что порядок строк идентичен порядку строк в файле формы¹⁹. Это означает, что мы можем просто добавить значения NDVI в файл формы в качестве нового столбца:

```
1 mw4$ndvi <- extractedndvi$NDVI
2 head(mw4)
```

	DIST_CODE	EA_CODE	TA_CODE	ndvi
1	105	10507801	10507	6225.000
2	105	10507072	10507	6624.000
3	105	10507010	10507	6112.167
4	105	10507001	10507	6883.500
5	105	10507009	10507	6684.200
6	105	10507033	10507	6252.667

Этот код берет колонку NDVI из данных `extractedndvi` и добавляет ее в файл формы `mw4` в качестве новой колонки `ndvi`. Результаты можно увидеть, просмотрев первые несколько строк файла `mw4`.

С этими данными мы можем визуализировать файл формы `admin4`, прямо дав команду функции `ggplot`, чтобы она окрасила объекты в соответствии с их значением `ndvi`:

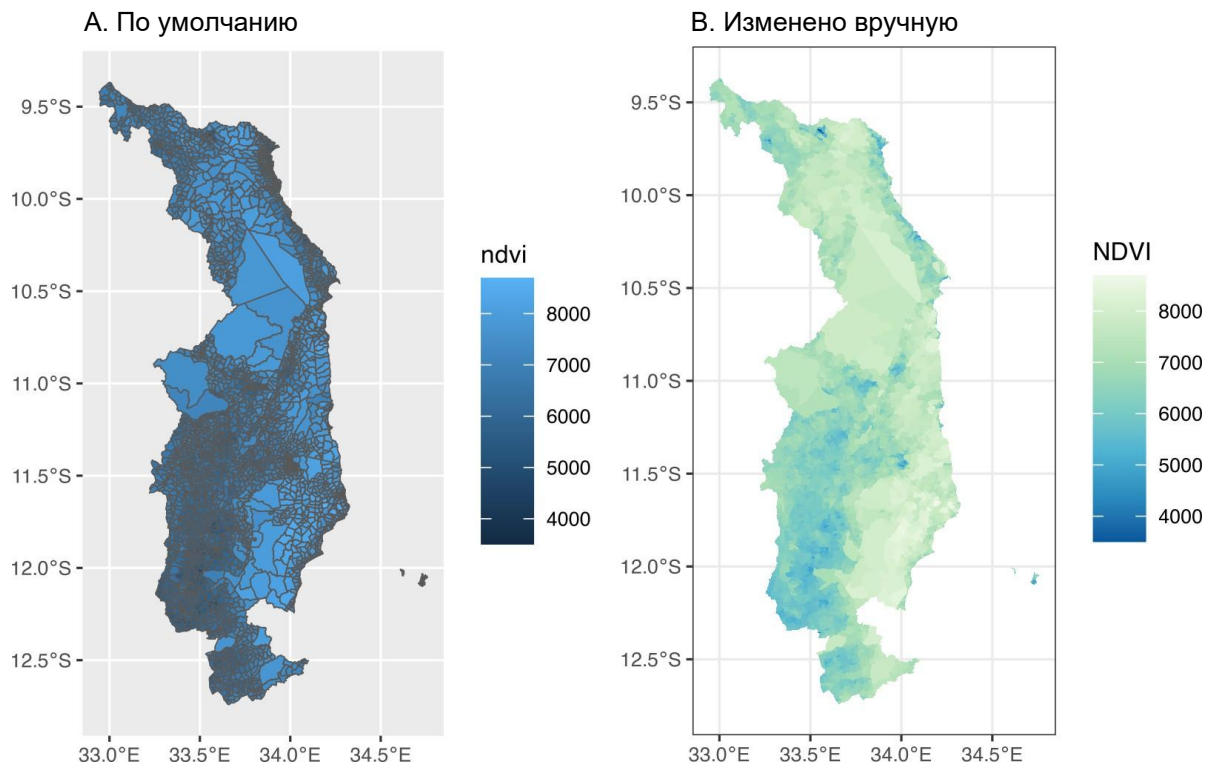
¹⁹ Мы используем функцию `head()`, чтобы вывести только первые шесть строк полученного объекта.

```

1 # Рисунок А
2 ggplot() +
3   geom_spatvector(data = mw4, aes(fill = ndvi))
4 # Рисунок В
5 ggplot() +
6   geom_spatvector(data = mw4, aes(fill = ndvi), color = NA) +
7   scale_fill_distiller("NDVI", palette = "GnBu") +
8   theme_bw(base_size = 8)

```

Рисунок 6: Файл формы admin4 со значениями индекса NDVI



На рис. А (слева) рисунка 6 показан файл формы admin4 со значениями индекса NDVI. Цветовая шкала – по умолчанию. На рис. В (справа) показан тот же файл формы, но с рядом изменений. Во-первых, поскольку наша цель – представить цвет *заливки* для каждого объекта уровня admin4, мы установили цвет контура каждого объекта как NA (т.е. прозрачный). Рис. А довольно трудночитаем из-за размера объектов относительно размера линий границ. Во-вторых, используем функцию `scale_fill_distiller` для изменения цветовой шкалы, снова применяя шкалу от зеленого к синему, которую мы использовали ранее. Наконец, меняем базовую тему на `theme_bw(base_size = 8)`. На наш взгляд, эти изменения приводят к тому, что иллюстрация становится гораздо более удобочитаемой.

4.3 Создание файла формы из точек

В папке `data` мы создали папку `ihshousehold`, которая содержит два отдельных набора данных на уровне домохозяйств из Пятого комплексного обследования домохозяйств (IHS5). Первый набор содержит агрегированные показатели потребления (т.е. расходы, а также показатель бедности), а второй – географические переменные домохозяйства, включая географические координаты²⁰. Пока что мы сосредоточимся на последних.

В целях конфиденциальности GPS-координаты в общедоступных наборах данных, как правило, не бывают точными; вместо этого они *изменяются* на случайную величину, чтобы сохранить анонимность респондентов. Другими словами, GPS-координаты – это не точное местоположение домохозяйства, а скорее случайно выбранная точка в пределах определенного расстояния от истинного местоположения. Кроме того, в данном наборе данных координаты относятся к счетному участку (первичной единице выборки) обследования, поэтому у многих домохозяйств координаты совпадают.

Наборы данных по домохозяйствам имеют расширение файла `.dta`, что означает, что это файлы программы Stata. Мы можем прочитать их в языке R с помощью пакета `haven` и функции `read_dta()` следующим образом:

```
1 # читаем файл .dta
2 df <- read_dta("data/ihshousehold/householdgeovariables_ihs5.dta")
3 colnames(df)
```

```
[1] "case_id"      "ea_id"         "dist_road"     "dist_agmrkt"
[5] "dist_auction" "dist_admarc"   "dist_border"   "dist_popcenter"
[9] "dist_boma"    "ssa_aez09"     "twi_mwi"       "sq1"
[13] "sq2"          "sq3"           "sq4"           "sq5"
[17] "sq6"          "sq7"           "af_bio_1_x"    "af_bio_8_x"
[21] "af_bio_12_x"  "af_bio_13_x"   "af_bio_16_x"   "afmnslp_pct"
[25] "srtm_1k"      "popdensity"    "cropshare"     "h2018_tot"
[29] "h2018_wetQstart" "h2018_wetQ"   "h2019_tot"     "h2019_wetQstart"
[33] "h2019_wetQ"   "anntot_avg"    "wetQ_avgstart" "wetQ_avg"
[37] "h2018_ndvi_avg" "h2018_ndvi_max" "h2019_ndvi_avg" "h2019_ndvi_max"
[41] "ndvi_avg"     "ndvi_max"      "ea_lat_mod"    "ea_lon_mod"
```

Новый объект `df` представляет собой набор данных на уровне домохозяйств с большим количеством переменных (44, если быть точным). В вышеприведенном коде используется функция `colnames(df)`, которая отображает названия столбцов (переменных) для того, чтобы мы могли определить, какие столбцы содержат GPS-координаты. В данном случае соответствующие столбцы для долготы и широты называются `ea_lon_mod` и `ea_lat_mod`, соответственно, где «ea» указывает на то, что они относятся к счетному участку, а не к домохозяйству, а «mod» указывает на то, что они модифицированы (искажены). Другой важный столбец – идентификатор домохозяйства, в данном случае это `case_id`. Поскольку нам действительно нужны только эти три переменные, мы можем использовать пакет `tidyverse` для выбора только этих столбцов:

²⁰ Наши данные ограничены данными по Северной провинции Малави.

```

1 # Выбираем нужные нам столбцы в df
2 <- df |>
3   select(case_id, ea_lon_mod, ea_lat_mod)
4 # простая сводная статистика
5 summary(df)

```

case_id	ea_lon_mod	ea_lat_mod
Length:2176	Min. :0.00	Min. : -12.610
Class :character	1st Qu.:33.50	1st Qu.: -11.482
Mode :character	Median :33.92	Median : -10.978
	Mean :32.09	Mean : -10.345
	3rd Qu.:34.03	3rd Qu.: -9.909
	Max. :34.72	Max. : 0.000
	NA's :1	NA's :1

Мы использовали функцию `select()` из пакета `tidyverse`, чтобы выбрать только нужные нам столбцы, используя оператор `pipe (|>)` для создания цепочки операций. Можно прочесть этот код так: «Возьми `df`, а затем выбери эти столбцы. Замени `df` новым объектом».

Функция `summary()` дает нам некоторую базовую сводную статистику по трем переменным. Тип значений переменной `case_id` – символьные (т.е. строка), а `ea_lon_mod` и `ea_lat_mod` – числовые. Однако, похоже, для этих двух переменных существуют некие непонятные минимальные/максимальные значения. Мы можем преобразовать эти данные в пространственный объект с помощью пакета `terra`, а затем визуализировать точки, чтобы рассмотреть их более внимательно.

```

1 # crs = "EPSG:4326" сообщает terra, что координаты точек являются долготой/широтой
2 df <- vect(df, geom = c("ea_lon_mod", "ea_lat_mod"), crs = "EPSG:4326")
3 df
4 ggplot() +
5   geom_spatvector(data = df) +
6   theme_bw(base_size = 8)

```

```

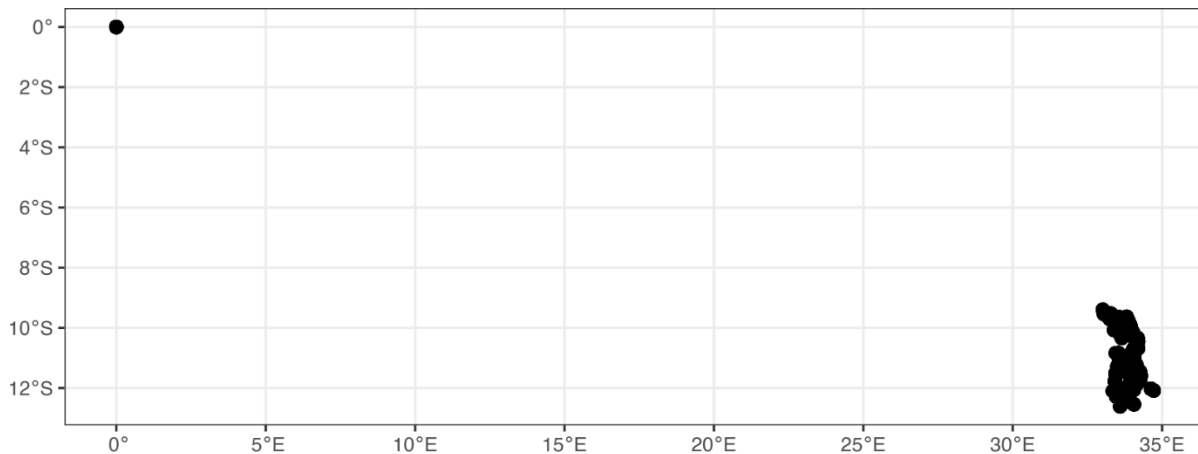
class       : SpatVector
geometry    : points
dimensions  : 2176, 1 (geometries, attributes)
extent      : 0, 34.71688, -12.60982, 0 (xmin, xmax, ymin, ymax)
coord.ref.  : lon/lat WGS 84 (EPSG:4326)
names       : case_id
type        : <chr>
values      : 101011000014
              101011000023
              101011000040

```

На рис. 7 хорошо видно, что в данных GPS есть несколько неверных значений! Некоторые домохозяйства имеют координаты (0, 0), чего не должно быть, так как эти домохозяйства явно находятся за пределами (Северной провинции) Малави. Поэтому пока мы просто удалим их из наших данных²¹:

²¹ Все домохозяйства с отсутствующими координатами (NA) были удалены при преобразовании данных в пространственный объект.

Рисунок 7: Местоположения домохозяйств в Северной провинции Малави



```

1 # функция geom(df) возвращает координаты
2 # аргумент [, "x"] сообщает: "дай мне координаты оси x"
3 # далее проверяем, не равны ли они нулю, и оставляем только те, которые не равны
4 df <- df[geom(df) [, "x"] != 0, ]
5 # теперь визуализируем
6 ggplot() +
7   geom_spatvector(data = df) +
8   theme_bw()

```

На рис. 8 видно, что мы удалили домохозяйства, координаты которых были указаны как (0, 0). Теперь мы можем использовать эти точки для извлечения информации из других файлов форм или других растров. Например, мы можем передать идентификаторы уровней `admin` из файла формы `mw4` в точки или получить значения индекса `NDVI` в местах расположения домохозяйств (или рядом с ними).

Мы можем извлечь информацию из другого *файла формы*, выполнив пространственное соединение с помощью пакета `terra`. Это можно сделать разными способами, но один из них заключается в использовании функции `extract()`, как показано ниже, при этом обратите внимание, что сначала идут полигоны, а затем точки:

```

1 dfmw4 <- extract(mw4, df)
2 dim(dfmw4)

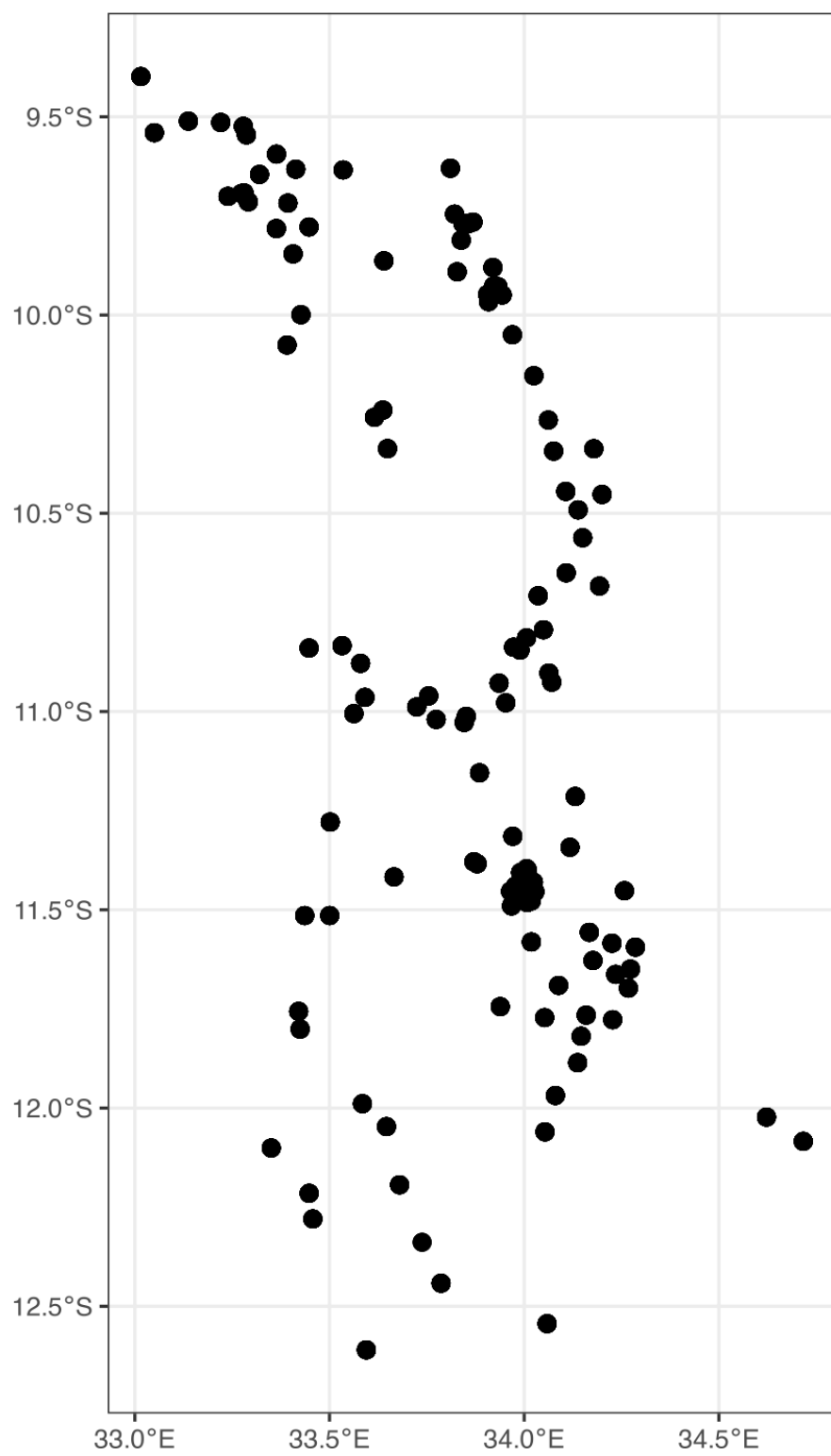
```

```
[1] 2063 5
```

```
1 dim(df)
```

```
[1] 2063 1
```

Рисунок 8: Очищенные местоположения в Северной провинции Малави



Если взглянуть на размеры (с помощью функции `dim()`), мы увидим, что новый объект имеет столько же строк, сколько и исходный объект `df`. Однако у него больше столбцов, поскольку он также содержит информацию из файла формы `mw4`. Ключевым уточнением является то, что новый объект `dfmw4` – это блок данных, а не пространственный объект. Другими словами, он не содержит никакой информации о пространственных координатах из исходного файла формы или растра, а представляет собой то, что вы получите, если, например, загрузите файл `.csv` в среду R. Впрочем, строки расположены в том же порядке, что и в исходном объекте `df`, поэтому мы можем просто добавить новые столбцы в исходный объект `df`:

```
1 # cbind, за исключением ПЕРВОГО столбца из объекта dfmw4
2 # первый столбец имеет имя "id.y", и он нам не нужен
3 df <- cbind(df, dfmw4[,-1])
4 head(df)
```

	case_id	DIST_CODE	EA_CODE	TA_CODE	ndvi
1	101011000014	101	10101006	10101	6098.6
2	101011000023	101	10101006	10101	6098.6
3	101011000040	101	10101006	10101	6098.6
4	101011000071	101	10101006	10101	6098.6
5	101011000095	101	10101006	10101	6098.6
6	101011000115	101	10101006	10101	6098.6

Поскольку мы уже извлекли индекс NDVI в файл формы `mw4`, теперь он также находится в новом объекте `df`. Но мы также можем извлечь растровые значения непосредственно в точки, снова используя функцию `extract()` и те же шаги, что и выше, создав новый столбец (или, в данном случае, заменив существующий столбец) под названием `ndvi`:

```
1 dfextracted <- extract(ndvi, df)
2 df$ndvi <- dfextracted$NDVI
```

Обратите внимание, что эти новые значения будут отличаться от тех, которые мы извлекли из файла формы `mw4`. Причина проста: файл формы `mw4` состоит из полигонов, и мы взяли *среднее* значение индекса NDVI для каждого полигона. Однако для точек домохозяйств мы извлекаем только растровое значение растровой сетки, в которой находится точка. Другой вариант – взять значения ближайших четырех растровых ячеек. Это можно сделать с помощью опции `method = "bilinear"` в функции `extract()`:

```
1 dfextracted <- extract(ndvi, df, method = "bilinear")
2 df$ndvibilinear <- dfextracted$NDVI
3 summary(df[[c("ndvi", "ndvibilinear")]])
```

	ndvi	ndvibilinear
Min.	:4442	Min. :4770
1st Qu.	:6033	1st Qu.:6116
Median	:6769	Median :6752
Mean	:6678	Mean :6683
3rd Qu.	:7233	3rd Qu.:7198
Max.	:8781	Max. :8538

Посредством функции `summary()` мы видим, что значения этих двух методов немного отличаются. В частности, в случае с опцией «`bilinear`» меньше экстремальных значений, поскольку мы берем среднее значение из многих растровых ячеек, а не из одной.

4.4 Создание сетки

Как уже упоминалось, альтернативой использованию административного деления для оценки малых районов является создание сетки, охватывающей Северную провинцию Малави. Напомним, что растр – это просто сетка, где все ячейки имеют одинаковый размер (разрешение). Мы хотим создать файл формы, который по сути является растром, поэтому сначала мы создадим растр, а затем преобразуем его в файл формы!

Мы можем создать растр с помощью функции `rast()` из пакета `terra`, но мы должны быть осторожны с разрешением. А именно, мы будем использовать файл формы для определения области, в которой мы хотим создать сетку. Мы должны указать разрешение в тех же единицах (CRS), что и файл формы. Если файл формы будет спроецирован, например, в системе UTM, то нам нужно указать разрешение в метрах. С другой стороны, если проекция будет иметь координаты долготы/широты, нам нужно указать разрешение в *градусах*. Поскольку наш файл формы `mw4` имеет координаты долготы/широты, для указания размера сетки мы будем использовать градусы²².

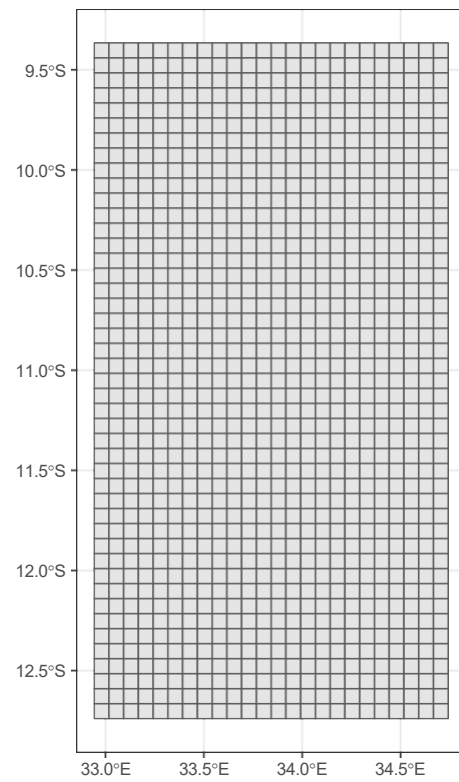
```
1 grid <- rast(mw4, res = 0.075)
2 grid <- as.polygons(grid)
3 grid$id <- 1:nrow(grid)
4 ggplot() +
5   geom_spatvector(data = grid) +
6   theme_bw(base_size = 8)
```

Что мы сделали? Мы создали растр с покрытием *охвата* файла формы `mw4`, каждая ячейка которого имеет разрешение 0,075 градуса. Затем мы преобразовали этот растр в файл формы с помощью функции `as.polygons()`. Столбец `id` – это просто уникальный идентификатор для каждой ячейки сетки, который мы будем использовать через минуту.

На рис. 9 показана созданная нами сетка, но тут есть проблема: поскольку она покрывает весь охват `mw4`, есть много ячеек сетки, которые лежат за пределами Северной провинции Малави. Нам нужно удалить все эти ячейки сетки из файла формы. Мы можем сделать это, найдя все ячейки сетки, которые перекрывают полигоны/объекты файла формы `mw4`, а затем отфильтровать файл формы `grid` так, чтобы в нем остались только те ячейки, которые перекрываются с файлом формы `mw4`. Именно здесь в игру вступает столбец `id`:

²² Вы можете не угадать с размером с первой попытки, особенно в градусах. Цель состоит в том, чтобы получить сетку, достаточно мелкую, чтобы учесть вариации в данных, но не настолько мелкую, чтобы модель получилась недостаточно хорошей.

Рисунок 9: Сетка для Северной провинции Малави

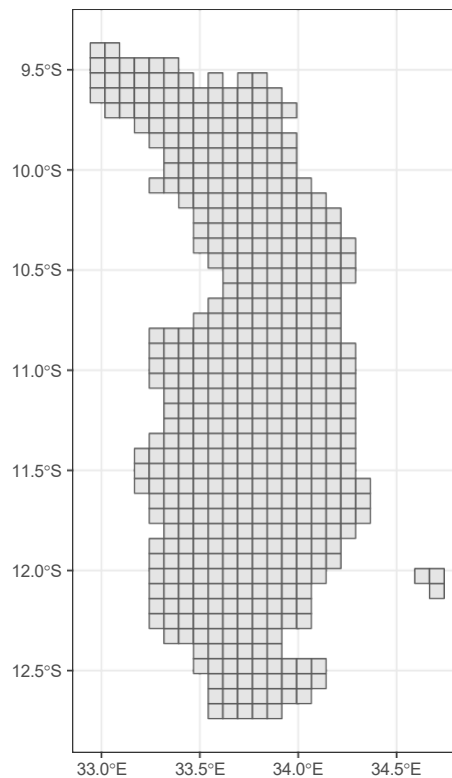


```

1 # создаем область пересечения
2 intersection <- intersect(grid, mw4)
3 # теперь отфильтровываем сетку с помощью столбца id
4 # (оставляем только те id, которые находятся в столбце id области пересечения)
5 grid <- grid |>
6   filter(id %in% intersection$id)
7 ggplot() +
8   geom_spatvector(data = grid) +
9   theme_bw(base_size = 8)

```

Рисунок 10: Отфильтрованная сетка для Северной провинции Малави



Функция `intersect()` создает новый файл формы, содержащий все пересечения между двумя файлами формы. В нем есть столбец `id` из файла формы `grid`, поэтому мы фильтруем файл формы `grid`, чтобы сохранить только те ячейки сетки, чей `id` находится в столбце `id` из области пересечения (`intersection`). На рис. 10 показана получившаяся сетка, которая теперь правильно отражает территорию Северной провинции Малави. Конечно же, теперь мы можем использовать этот файл формы с сеткой для извлечения растровых данных, как мы делали раньше. Мы также можем найти ячейку сетки, в которую попадают домохозяйства, используя тот же синтаксис, что и выше, когда мы помещали домохозяйства в административные районы, чтобы извлечь идентификатор `admin4`.

4.5 Где мы можем найти растры?

Растры можно найти относительно легко, но доступ к ним не всегда прост. Тем не менее, есть несколько веб-сайтов, на которых размещены простые растровые файлы (напр., файлы .tif или .nc), которые можно легко скачать. К ним относятся следующие:

- **WorldPop**: WorldPop предоставляет расчетные данные о численности населения в растровой форме почти для всех стран мира. Данные доступны с разным разрешением (напр., 100 м и 1 км) и за разные годы.
- **TerraClimate**: На сайте TerraClimate представлены ежемесячные оценки различных климатических переменных, включая осадки и температуру. Вы можете скачать данные прямо с сайта, перейдя по пунктам меню Download > Individual Years, выбрав нужный год и щелкнув по ссылке HTTPServer.
- **Colorado School of Mines - Nighttime Lights**: Здесь вы найдете ежемесячные и ежегодные комбинированные изображения ночной освещенности, доступные для бесплатного скачивания.
- **Mosaiks**: Mosaiks предоставляет доступ к объектам (растрам), которые были созданы с помощью машинного обучения и спутниковых снимков (Rolf et al. 2021). За подробностями следует обращаться к веб-сайту и соответствующему документу, но судя по опыту, эти объекты нередко могут достаточно хорошо предсказывать многие результаты. Весь набор данных очень велик, поэтому рекомендуется загружать только те пространственные области, которые вам нужны.
- **Open Buildings**: Проект Open Buildings предоставляет доступ к горизонтальным проекциям зданий для многих стран, хотя это и не растры. Эти проекции представляют собой файлы форм. Количество зданий может в значительной степени предсказывать многие результаты, в том числе уровни бедности. Следует помнить, что файлы форм могут быть *очень* большими, поэтому работа с ними может быть затруднена из-за нехватки компьютерной памяти.

Ниже мы кратко и более подробно рассмотрим два проекта из перечисленных.

4.6 Worldpop

WorldPop предлагает (смоделированные/оценочные) данные о численности населения на дезагрегированном уровне почти для всех стран. Данные доступны с разным разрешением (напр., 100 м и 1 км) и за разные годы, но на момент написания данного руководства последним доступным годом является 2020 г. Данные Worldpop – это, вероятно, самые простые в доступе данные из всех растровых данных, рассматриваемых здесь. Вы можете скачать данные непосредственно с [веб-сайта](#), выбрав методику оценки (безусловную или условную)²³ и разрешение (100 м или 1 км), а затем выполнив поиск по стране и году, интересующим вас. После этого вы можете напрямую скачать файл .tif, который можно прочитать в среде R с помощью описанных выше методов.

²³ Согласно веб-сайту WorldPop, существующие два метода оценки являются следующими: «1. Оценка по всем квадратам наземной сетки в глобальном масштабе (безусловная); и 2. Оценка только в пределах территорий, на которых отмечены населенные пункты (условная)».

4.7 Mosaiks

Mosaiks предоставляет доступ к объектам (растрам), созданным с помощью машинного обучения и спутниковых снимков (Rolf et al. 2021). Весь набор данных очень велик, поэтому настоятельно рекомендуется загружать только те пространственные области, которые вам нужны. Чтобы получить доступ к необработанным данным, необходимо создать учетную запись на [веб-сайте](#). После входа в систему вы можете кликнуть по опции Map Query в верхней части страницы, а затем использовать координаты для ограничительной рамки, чтобы скачать только те данные, которые вам нужны.

Доступ к данным может быть несколько затруднен из-за ограничений на объем данных, которые можно скачать за один раз. Поэтому для больших территорий вам придется загружать данные небольшими порциями. Однако есть и альтернативный вариант. Это приведет к получению чуть менее точных данных, но это будет гораздо быстрее и может обеспечить приемлемую прогностическую значимость. В верхней части веб-сайта есть опция File Query. Если вы выберете ее и предоставите файл .csv с координатами каждой интересующей вас административной области, вы получите данные, связанные с этими координатами. Например, вы можете создать «центроиды» для нашего файла формы уровня admin4 (СУ) для Малави следующим образом:

```
1 mw4 <- vect("data/mw4.shp")
2 # создаем центроиды
3 mw4centroids <- centroids(mw4)
4 # создаем столбцы для широты (Latitude) и долготы (Longitude):
5 mw4centroids$Latitude <- geom(mw4centroids)[,"y"] # столбец "lat" (широта)
6 mw4centroids$Longitude <- geom(mw4centroids)[,"x"] # столбец "lon" (долгота)
7 # оставляем только то, что нам нужно
8 mw4centroids <- as_tibble(mw4centroids) |>
9   select(Latitude, Longitude, EA_CODE)
10 # сохраняем
11 write_csv(mw4centroids, "data/mw4centroids.csv")
12 # смотрим, как это выглядит
13 head(mw4centroids)
```



```
# A tibble: 6 x 3
  Latitude Longitude EA_CODE
  <dbl>      <dbl>   <chr>
1  -12.6       33.6  10507801
2  -12.7       33.7  10507072
3  -12.5       33.6  10507010
4  -12.4       33.7  10507001
5  -12.5       33.6  10507009
6  -12.5       33.6  10507033
```

Вы можете загрузить этот файл .csv на страницу File Query веб-сайта Mosaiks и через некоторое время найти полученные данные на странице My Files (Мои файлы). Обратите внимание, что вам не обязательно называть столбцы точно так же, как в примере выше; веб-сайт позволяет вам выбрать правильные столбцы для координат x и y. Однако, чтобы упростить задачу, в приведенном выше коде используются те же названия и порядок столбцов, что и в примере на веб-сайте.

4.8 Использование языка Python для доступа к сервису Google Earth Engine

Несмотря на существование множества способов доступа к растровым данным, вероятно, самой большой коллекцией наборов данных является [Google Earth Engine \(GEE\)](#) ([Gorelick et al. 2017](#)). GEE – это хранилище данных, содержащее множество различных типов растровых данных, включая спутниковые снимки, климатические данные, индексы растительности, классификацию земель и многое другое.

К сожалению, скачать данные с GEE не так просто, как с одного из перечисленных выше веб-сайтов. На веб-сайте GEE есть редактор кода, но он может быть не очень удобен для пользователей, не знакомых с JavaScript. Существует также API²⁴, но он работает на языке Python. Несмотря на наличие пакета R, он является лишь оболочкой для Python, то есть вам все равно придется скачать и установить Python на свой компьютер. Кроме того, запуск Python внутри R (с помощью пакета `reticulate`) сопряжен с определенными трудностями. Поэтому для целей данного руководства для доступа к GEE рекомендуется использовать Python напрямую.

Один из вариантов – просто использовать Python на своем компьютере. Однако мы считаем, что это сопряжено со своими трудностями, поскольку установка Python с нуля и его настройка может оказаться непростым делом для тех, кто не имеет опыта работы с ним. Вместо этого мы рекомендуем пользователям использовать [Google Colab](#) – бесплатный сервис, позволяющий запускать код Python в облаке. Преимуществом использования Google Colab является отсутствие необходимости устанавливать Python на свой компьютер (а также возможность доступа к данным из любого места, где есть интернет, что, конечно, может быть и недостатком).

Для начала мы загрузили в репозиторий GitHub «блокнот» на языке Python, в котором содержится информация о том, как загружать данные; это файл `geospatialpull.ipynb`. Чтобы начать работу с ним, вам нужно создать аккаунт на сервисе Google Earth Engine. На [Главной странице GEE](#) щелкните по опции *Get Started* (Начать) в правом верхнем углу. Чтобы войти в систему, вам потребуется учетная запись Google, затем нужно будет выполнить шаги по регистрации некоммерческого проекта, созданию нового «Облачного проекта Google» (Google Cloud Project) и включению API Earth Engine. После выполнения шагов по созданию «Облачного проекта» вы сможете получить доступ к API Earth Engine. *Не забудьте записать название проекта – оно понадобится вам позже.*

После создания аккаунта GEE вы можете скопировать блокнот `geospatialpull.ipynb` в сервис Google Colab. Это можно сделать, открыв [веб-сайт Google Colab](#) и щелкнув по пункту меню Файл > Создать блокнот на Диске. Затем вы можете скопировать и вставить код из блокнота `geospatialpull.ipynb` в блокнот Google Colab. Вам понадобится учетная запись Google, но она уже должна быть у вас, если вы следовали инструкциям выше по созданию аккаунта GEE; используйте эту же учетную запись для входа в Google Colab, если вас попросят.

Вам нужно будет подтвердить подлинность своего аккаунта Google, выполнив строки 14-20 в примере скрипта (после копирования всего скрипта в блокнот Google Colab), что позволит вам получить доступ к API GEE. Для этого нужно выделить все соответствующие

²⁴ На [веб-сайте IBM](#) можно найти отличное введение в API.

строки и нажать на клавиатуре Control + Enter (или Command + Enter на Mac). Вам будет предложено разрешить доступ к вашей учетной записи GEE, что следует разрешить.

4.8.1 Использование скрипта на языке Python

Работа с блокнотом `geospatialpull.ipynb` включает в себя несколько этапов. Во-первых, вам необходимо пройти аутентификацию API с помощью функции `ee.Authenticate()` (как было описано выше). При этом в браузере откроется новая вкладка, где вам будет предложено войти в свою учетную запись Google и разрешить API доступ к вашей учетной записи. После выполнения этих шагов вы увидите ключ API, который нужно скопировать. Затем вы можете вставить этот ключ в терминале и нажать клавишу Enter, что позволит вам получить доступ к API.

Во-вторых, необходимо инициализировать API с помощью функции `ee.Initialize()`. Нужно указать имя облачного проекта, о котором говорилось выше, следующим образом:

```
1 ee.Authenticate()
2 ee.Initialize(project="NAME")
```

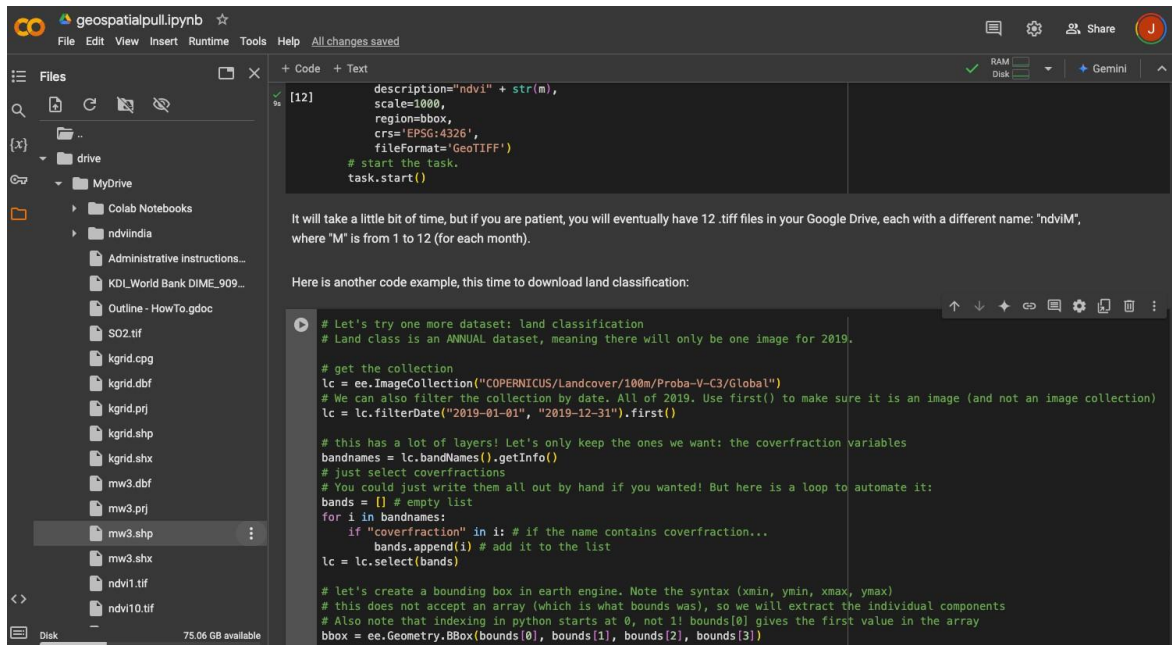
где вместо "NAME" должно быть название вашего облачного проекта. Другими словами, вам нужно будет изменить скрипт в Google Colab, чтобы включить в него название вашего облачного проекта.

Одной из полезных функций Google Colab является возможность скачать файл формы на ваш Диск Google, доступ к которому вы получили после создания учетной записи Google. Вы также можете скачать его на свой диск непосредственно через Google Colab. В крайней левой части экрана есть пять иконок, расположенных вертикально; первая выглядит как три пункта списка, а последняя – как папка. Щелкните по иконке папки. Чуть выше появятся четыре горизонтальных иконки; третья иконка выглядит как папка с треугольным значком Диска Google на ней (если навести на нее указатель мыши, всплывет подсказка «Подключить Диск»). Щелкните по ней. Затем Google предложит вам подключить ваш Диск вручную; просто следуйте инструкциям и предоставляйте все запрашиваемые разрешения. Наконец, после того как Диск будет «подключен», щелкните по папке «drive», а затем по папке «MyDrive». При наведении указателя мыши на папку «MyDrive» вы увидите три точки с правой стороны. Щелкните по ним, затем выберите «Скачать» и скачайте все файлы, связанные с вашим файлом формы. Другими словами, для Малави вы должны скачать все файлы `mw3`, а не только `mw3.shp`. В данном случае имеется четыре отдельных файла: `mw3.shp`, `mw3.shx`, `mw3.dbf` и `mw3.prj`.

В нашем примере мы используем файл формы для Малави. Если вы уже скачали файл формы, вы можете пропустить большинство шагов, описанных выше, и просто «подключить» диск в Google Colab, используя код, который уже есть в блокноте:

```
1 from google.colab import drive
2 drive.mount('/content/drive')
```

Рисунок 11: Подключение Диска Google в сервисе Colab



Мы поместили наш файл формы (`mw3.shp`) в нашу папку Диска Google. После выполнения приведенного выше кода (щелчком по кнопке «play» в левом верхнем углу фрагмента кода) вы сможете увидеть свой Диск Google в левой части экрана, как показано на рис. 11.

Если щелкнуть по трем точкам, в появившемся меню можно выбрать пункт «Скопировать путь», чтобы получить путь к файлу формы. Затем вы можете использовать этот путь в скрипте Python для загрузки файла формы в Python с помощью библиотеки «geopandas». Вот код для указания расположения файла `mw3.shp` в моем Диске Google (возможно, вам придется изменить путь в зависимости от расположения вашего файла формы, который вы скопировали с помощью команды «Скопировать путь»):

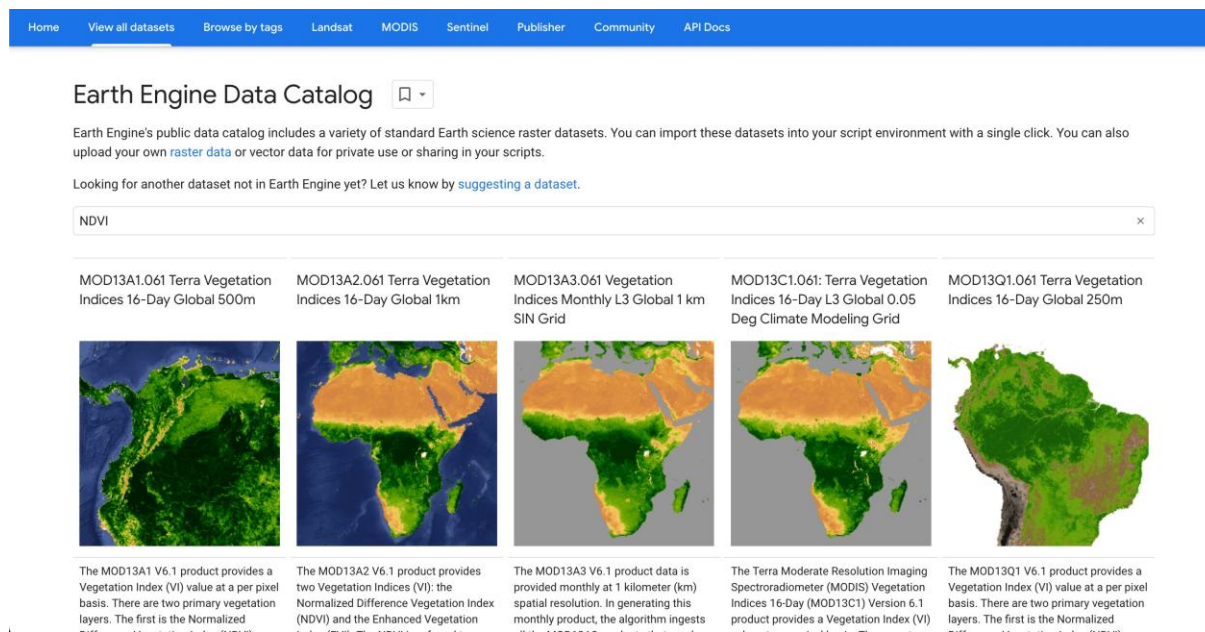
```
1 shape = geopandas.read_file("/content/drive/MyDrive/mw3.shp")
2 # убедимся, что координаты указаны для широты/долготы (спроецируем их)
3 shape = shape.to_crs("EPSG:4326")
4 # получим общие границы для файла формы
5 bounds = shape.total_bounds
```

Помимо загрузки файла формы в Python, код делает еще две вещи. Во-первых, он преобразует координаты файла формы в долготу/широту («EPSG:4326», что мы уже видели выше). Это нужно для того, чтобы создать правильную «рамку» для нашей области. Затем он находит «общие границы» фигуры, то есть рамку, которая полностью содержит файл формы (см. рис. A1 в приложении). Мы будем использовать эту рамку для определения области, из которой мы хотим скачать данные.

Теперь нужно решить, какой набор данных GEE мы хотим скачать. В качестве примера возьмем индекс NDVI. Перейдите на [главную страницу GEE](#) и щелкните на «Datasets» в

верхней части страницы (примерно посередине). На следующей странице щелкните «View all datasets». На [следующей странице](#) мы можем искать наборы данных, используя ключевые слова. Наберите в поисковой строке «NDVI» (без кавычек) и нажмите клавишу Enter. Здесь вы увидите множество результатов поиска, как показано на рис. 12.

Рисунок 12: Результаты поиска по базе данных Google Earth Engine



Скачайте данные о ежемесячном индексе NDVI с разрешением 1 км, это третий вариант на рисунке выше. Щелкните по этому набору данных, который называется «MOD13A3.061 Vegetation Indices Monthly L3 Global 1 km SIN Grid». На следующей странице вы увидите описание данных. Вот некоторые из важных сведений на этой странице:

- **Dataset Availability:** Здесь отображаются даты, для которых доступны данные. В этом случае данные доступны с февраля 2000 года по настоящее время (или, по крайней мере, за месяц до настоящего времени).
- **Dataset Provider:** Откуда получены данные.
- **Earth Engine Snippet:** Это очень важные сведения. Это идентификатор, который вы будете использовать для доступа к данным в Python. Для данного набора данных идентификатором является MODIS/061/MOD13A3.
- **Bands:** Сразу под разделом «Tags» (Теги) находится список различных вкладок. На вкладке «Bands» (Каналы) показаны:
 - Разрешение раstra. Мы выбрали вариант «1 км», поэтому его разрешение составляет 1000 метров.
 - Название каналов. В данном случае есть два канала индексов растительности: NDVI и EVI. Мы пока остановимся только на канале NDVI.
 - Минимальное и максимальное значения каналов. Значения NDVI должны находиться в диапазоне от -1 до 1, и мы можем получить его, умножив

минимальное/максимальное значение на масштаб (в данном случае 0,0001). Однако, учитывая то, что мы хотим сделать с этими данными, мы можем просто оставить их исходные значения.

Теперь попробуем получить доступ к этому набору данных в Python. Для доступа к данным можно использовать функцию `ee.ImageCollection`. Код выглядит следующим образом:

```
1 # посмотрим на индекс NDVI
2 ndvi = ee.ImageCollection("MODIS/061/MOD13A3")
3 print(ndvi.getInfo())
```

Это дает API команду на осуществление доступа к указанному набору данных, который является «коллекцией изображений» (Image Collection). Команда `print(ndvi.getInfo())` выводит информацию в консоль. В данном случае она выводит *много* информации, поэтому мы не будем приводить ее ниже. Если информация будет выведена, вы будете знать, что успешно выполнили запрос о доступе к набору данных и можете переходить к следующему шагу.

Этот набор данных охватывает более 20 лет. Данные по домохозяйствам, которые мы используем, относятся к 2019 году, поэтому сейчас мы будем загружать только данные за 2019 год²⁵. Для начала возьмем только январь 2019 года. Это можно сделать с помощью фильтрации объекта `ndvi`, как показано ниже:

```
1 ndvi = ndvi.filterDate("2019-01-01", "2019-01-31")
```

Теперь у нас есть отфильтрованный набор данных, содержащий данные за январь 2019 года. Далее мы сделаем еще две вещи. Во-первых, мы выберем только данные по NDVI (т.е. избавимся от EVI). Во-вторых, мы возьмем «среднее значение» изображения (которое на самом деле является средним значением каждой ячейки в растре), чтобы убедиться, что у нас есть только одно изображение, а не коллекция изображений. Код для этого выглядит следующим образом:

```
1 # для объектов, имеющих много каналов (растровых слоев), мы можем выбрать только те,
  ↳ которые нам нужны:
2 ndvi =
3 ndvi.select("NDVI")
```

```
<ee.imagecollection.ImageCollection object at 0x369d75490>
```

```
1 # наконец, убедимся, что у нас именно ОДНО изображение, а не коллекция изображений
2 ndvi = ndvi.mean()
3 ndvi
```

```
<ee.image.Image object at 0x36ac7c320>
```

²⁵ В теории мы также можем вывести долгосрочные средние значения или стандартные отклонения, что потребует извлечения данных и за другие годы. Однако в целях экономии времени мы сосредоточимся только на 2019 годе.

Мы видим, как изменился вывод данных после вызова функции `ndvi.mean()`. До ее вызова объект `ndvi` имел класс `ee.imagecollection.ImageCollection`. После вызова функции его класс сменился на `ee.image.Image`. Это то, что нам нужно, поскольку теперь мы можем скачать данные.

Мы также хотим выбрать данные только для определенной части, а не для всего земного шара, чтобы сделать код быстрее и уменьшить размер результирующего раstra. Мы уже создали ограничительную рамку для Северной провинции Малави, но необходимо внести одно изменение, чтобы GEE интерпретировал рамку правильно. Мы собираемся создать объект `ee.Geometry.Bbox`. Для этого нам нужно задать ему четыре значения:

1. Минимальную долготу
2. Минимальную широту
3. Максимальную долготу
4. Максимальную широту

Он не примет объект типа `array` (массив), который мы создали ранее, поэтому мы создадим геометрический объект следующим образом²⁶:

```
1  """
2  Давайте создадим ограничительную рамку в Google Earth Engine.
3  Обратите внимание на синтаксис (xmin, ymin, xmax, ymax)
4  Объект типа массив (array) не будет принят (он использовался для рамки),
5  поэтому мы извлечем отдельные компоненты
6  Также обратите внимание на то, что индексирование в языке python
7  начинается с 0, а не с 1! bounds[0] выдает первое значение из массива
8  """
9  bbox = ee.Geometry.BBox(bounds[0], bounds[1], bounds[2], bounds[3])
```

Теперь мы можем отправить объект `bbox` с нашим кодом на выполнение в GEE и получить только те данные, которые будут содержаться внутри ограничительной рамки.

Вот код для запуска скачивания с последующим описанием:

```
1  task = ee.batch.Export.image.toDrive(image=ndvi,
2      description='ndvi1',
3      scale=1000,
4      region=bbox,
5      crs='EPSG:4326',
6      fileFormat='GeoTIFF'
7  ') task.start()
8  task.status()
```

- ① `image=ndvi`: Это изображение, которое мы хотим скачать. В данном случае это среднее значение индекса NDVI за январь 2019 года (что мы и указали ранее до вызова этой функции).

²⁶ В языке Python используется так называемая «нулевая индексация». Это означает, что первый элемент в объекте Python обозначается как 0, а не 1. В этом его отличие от языка R.

- ② `description='ndv11'`: Это имя файла, который будет скачан. Вы можете изменить его на любое другое.
- ③ `scale=1000`: Это разрешение растра. Данный набор данных GEE имеет разрешение 1000 м, поэтому здесь имеет смысл выбрать такое же. Никогда не имеет смысла выбирать разрешение *выше* (т.е. мельче), чем исходное, но можно выбрать разрешение ниже (т.е. крупнее), например, чтобы экспортировать объект меньшего размера.
- ④ `region=bbox`: Это область земного шара, которую мы хотим скачать. Мы создали нашу рамку на основе Северной провинции Малави.
- ⑤ `crs='EPSG:4326'`: Указывает на тип CRS для скачиваемого файла. В этом примере система координат – долгота/широта.
- ⑥ `fileFormat='GeoTIFF'`: Формат файла, который мы хотим скачать. Поскольку это растр, мы скачаем файл в формате GeoTIFF (файл `.tif`).
- ⑦ `task.start()`: Это запустит скачивание. Вы должны выполнить эту строку кода, чтобы фактически скачать данные.
- ⑧ (Необязательно) `task.status()`: Позволяет видеть статус скачивания. В зависимости от размера данных весь процесс иногда может занять некоторое время. В данном примере он должен быть относительно быстрым.

По окончании выполнения задания полученный растр (в данном случае файл `ndv11.tif`) будет сохранен на Диске Google, связанном с учетной записью, которую вы использовали для входа в GEE. Бесплатная версия Диска Google предоставляет всего 1 ГБ хранилища, поэтому будьте внимательны при его использовании. Рекомендуется перемещать свои файлы `.tif` с Диска Google на компьютер сразу после завершения их скачивания.

В блокноте `geospatialpull.ipynb` также есть пример того, как скачать растры для всех месяцев 2019 года с помощью цикла `for loop`. Ознакомьтесь со строками 70-101 в этом скрипте.

Вы можете проделать те же шаги для любого другого показателя. Здесь мы приводим еще один пример для почвенно-растительного покрова, который сильно коррелирует с уровнем урбанизации и, как следствие, с уровнем бедности. Мы будем использовать идентификатор набора данных `COPERNICUS/Landcover/100m/Proba-V-C3/Global`, который предоставляет *ежегодные* оценки почвенно-растительного покрова с разрешением 100 м. Информацию об этом наборе данных можно найти на сайте GEE [здесь](#).

Как и раньше, мы загружаем набор данных, фильтруем его под 2019 год (в данном случае мы будем фильтровать под весь 2019 год, а не только январь), выбираем нужные нам каналы, а затем скачиваем его. Код выглядит следующим образом:

```
1 # Получим коллекцию
2 lc = ee.ImageCollection("COPERNICUS/Landcover/100m/Proba-V-C3/Global")
3 # Мы также можем отфильтровать коллекцию по дате. Весь 2019 год.
4 lc = lc.filterDate("2019-01-01", "2019-12-31").first()
```

Обратите внимание на применение метода `.first()` на четвертой строке. Так мы убедимся, что функция возвращает изображение, а не коллекцию изображений. Затем мы

можем выбрать нужные нам каналы и скачать данные, как мы это делали в случае с NDVI. На странице GEE для этого набора данных видно, что он содержит множество различных каналов. Мы скачаем все каналы, содержащие слово «coverfraction». Чтобы получить названия каналов, мы можем использовать метод `.bandNames().getInfo()`:

```
1 bandnames = lc.bandNames().getInfo()
2 bandnames

['discrete_classification', 'discrete_classification-proba', 'bare-coverfraction',
'urban-coverfraction', 'crops-coverfraction', 'grass-coverfraction',
'moss-coverfraction', 'water-permanent-coverfraction',
'water-seasonal-coverfraction', 'shrub-coverfraction', 'snow-coverfraction',
'tree-coverfraction', 'forest_type', 'data-density-indicator', 'change-confidence']
```

Теперь мы хотим выбрать *только* те названия каналов, которые содержат строку «coverfraction». Сделать это можно несколькими способами. Один из них – просто указать, что нужные нам каналы – это каналы с 3 по 12 (из-за особенностей Python они индексируются как с 2 по 11). Затем мы можем использовать функцию `ee.Image.select()`, чтобы выбрать только эти каналы, не забывая о том, что в Python элементы в списке вызываются иначе, чем в языке R:

```
1 lc = lc.select(bandnames[2:12])
2 # перепроверяем
3 lc.bandNames().getInfo()

['bare-coverfraction', 'urban-coverfraction', 'crops-coverfraction',
'grass-coverfraction', 'moss-coverfraction', 'water-permanent-coverfraction',
'water-seasonal-coverfraction', 'shrub-coverfraction', 'snow-coverfraction',
'tree-coverfraction']
```

В блокноте `geospatialpull.ipynb` показан еще один пример, который должен помочь вам лучше познакомиться с циклами в Python. В скрипте Python цикл находится на строках 144 – 156.

После выбора необходимых каналов вы можете скачать данные, используя тот же синтаксис, что и раньше (вместе с той же ограничительной рамкой):

```
1 task = ee.batch.Export.image.toDrive(image=lc,
2   description='lc',
3   scale=100, # теперь 100, поскольку таково разрешение данных
4   region=bbox,
5   crs='EPSG:4326',
6   fileFormat='GeoTIFF')
7 task.start()
8 task.status()
```

Обратите внимание, что скачивание данных займет больше времени, чем в случае с данными индекса NDVI, поскольку вы загружаете несколько каналов с более высоким

разрешением. После скачивания данных они снова появятся в вашем Диске Google. Переместите их в папку на своем компьютере, в которой вы работали.

4.9 Проект Geolink

На момент написания данного руководства в разработке находится пакет, позволяющий получать геопространственные данные непосредственно в среде языка R, без использования Google Earth Engine или Python. Документацию и примеры можно найти на [GitHub](#). Этот пакет все еще находится в разработке, поэтому мы рекомендуем читателям следить за обновлениями на GitHub.

4.10 Заключительный этап

На этом этапе вы должны уже скачать все геопространственные данные, необходимые для модели OMP. Важной частью этих данных является оценка численности населения (из базы данных WorldPop), которую мы будем использовать позже в качестве весов. Некоторые распространенные наборы данных, используемые для OMP, включают:

- Численность населения
- Индекс NDVI (помесячный)
- Ночная освещенность
- Классификация почвенно-растительного покрова
- Mosaiks

Разумеется, вы можете добавить все остальное, что, по вашему мнению, может оказаться полезным. Мы же будем использовать только вышеуказанные данные в остальной части этого руководства²⁷. Итоговые переменные на уровне СУ (admin4) сохранены в папке `finalgeovars` репозитория GitHub. Если вы хотите посмотреть, как эти переменные выглядят в R, обратитесь к скрипту `geoaggregation.R` в репозитории GitHub. Итоговый набор данных, который мы будем использовать в этом руководстве, находится по пути `data/geovarseas.csv`.

²⁷ В выложенные на GitHub данные мы включили только одно подмножество объектов Mosaiks из-за их размера. Если у вас достаточно оперативной памяти, вы можете загрузить и включить все из них.

5 Данные обследования

Мы уже видели данные обследования домохозяйств, когда смотрели на GPS-координаты домохозяйств. Однако прежде чем продолжить, нам необходимо выполнить еще несколько шагов: нужно объединить данные GPS с данными о бедности, а затем агрегировать их до уровня admin 4 для проведения оценки модели ОМР.

5.1 Подготовка данных обследования

Первым делом мы загружаем GPS-данные домохозяйств и данные о бедности²⁸, удаляем домохозяйства с отсутствующими координатами или с координатами (0, 0), а затем соединяем два набора данных домохозяйств вместе.

```
1 # загружаем оба набора данных
2 df <- read_dta("data/ihshousehold/householdgeovariables_ihs5.dta")
3 pov <- read_dta("data/ihshousehold/ihs5_consumption_aggregate.dta")
4 # удаляем из df отсутствующие и нулевые координаты
5 df <- df |>
6   filter(!is.na(ea_lon_mod), ea_lon_mod!=0)
7 # оставляем только то, что нам нужно
8 df <- df |>
9   select(case_id, ea_lon_mod, ea_lat_mod)
10 pov <- pov |>
11   select(case_id, hhsize, hh_wgt, poor)
12 # теперь соединяем pov и df
13 df <- df |>
14   left_join(pov, by = "case_id")
15 head(df)
```

```
# A tibble: 6 x 6
  case_id      ea_lon_mod ea_lat_mod hhsize hh_wgt poor
  <chr>      <dbl>     <dbl>  <dbl>  <dbl> <dbl>
1 101011000014      33.2      -9.70     4    93.7     1
2 101011000023      33.2      -9.70     4    93.7     0
3 101011000040      33.2      -9.70     4    93.7     1
4 101011000071      33.2      -9.70     5    93.7     0
5 101011000095      33.2      -9.70     5    93.7     0
6 101011000115      33.2      -9.70     2    93.7     0
```

Мы использовали функцию `left_join()` из пакета `tidyverse`, чтобы соединить два набора данных вместе. Теперь нам нужно преобразовать наш новый объект в пространственный объект, что мы можем сделать с помощью пакета `terra` (как мы делали в разделе 4.3). Затем мы извлечем информацию из файла формы `mw4`.

²⁸Необработанные данные обследования также включают информацию о расходах, которую НСО Малави использует для расчета показателей бедности для каждого домохозяйства.

```

1 # преобразуем в пространственный объект
2 df <- vect(df, geom = c("ea_lon_mod", "ea_lat_mod"), crs = "EPSG:4326")
3 # загружаем mw4
4 mw4 <- vect("data/mw4.shp")
5 # убеждаемся, что у них одна и та же система CRS
6 mw4 <- project(mw4, crs(df))
7 # извлекаем информацию
8 extracted <- extract(mw4, df)
9 # добавляем к df, за исключением первого столбца
10 df <- cbind(df, extracted[,-1])
11 head(df)

```

	case_id	hhsize	hh_wgt	poor	DIST_CODE	EA_CODE	TA_CODE
1	101011000014	4	93.7194	1	101	10101006	10101
2	101011000023	4	93.7194	0	101	10101006	10101
3	101011000040	4	93.7194	1	101	10101006	10101
4	101011000071	5	93.7194	0	101	10101006	10101
5	101011000095	5	93.7194	0	101	10101006	10101
6	101011000115	2	93.7194	0	101	10101006	10101

Теперь у нас есть данные по домохозяйствам, которые включают размер домохозяйства, веса домохозяйства, показатель бедности, код ТВ (код admin3) и код СУ (код admin4). Теперь давайте агрегируем данные до уровня admin4:

```

1 df <- as_tibble(df) |> # это выносит df из «пространственного» объекта
2   group_by(EA_CODE, TA_CODE) |> # это идентификаторы admin4 и admin3
3 # функция summarize агрегирует до уровня СУ/ТВ (также и СУ)
4   summarize(poor = weighted.mean(poor, hhsize*hh_wgt),
5             total_weights = sum(hhsize*hh_wgt)) |>
6   ungroup()
7 head(df)

```

```

# A tibble: 6 x 4
  EA_CODE TA_CODE poor total_weights
  <chr>   <chr>   <dbl>      <dbl>
1 10101006 10101  0.243      6560.
2 10101011 10101  0.468      8953.
3 10101027 10101  0.0959     10582.
4 10101033 10101  0.384      8571.
5 10101039 10101  0.591     10612.
6 10101054 10101  0.494      6211.

```

Наш новый объект df теперь имеет уровень admin4 и содержит средние уровни бедности, общие веса домохозяйств и идентификаторы для уровней admin3 и admin4. Он будет служить «выборкой» для нашей модели малых районов.

Что нам нужно? Нам нужен набор данных с:

- Идентификатором уровня admin
- Интересующим нас результатом (напр., расходами)

Затем мы можем объединить их с геопространственными данными, в данном случае на уровне admin4, чтобы провести оценку модели ОМР.

6 Создание и выбор объектов

Теперь, когда у нас есть все геопространственные объекты, нам нужно подумать о том, какие объекты мы хотим использовать в нашей модели и хотим ли мы использовать наши необработанные объекты для создания новых показателей. Выбор объектов особенно важен при работе с геопространственными данными из-за огромного количества объектов, которые мы можем создать. Мы уже создали ряд объектов, но мы также можем создавать новые объекты, преобразовывать существующие и использовать метод лассо-регрессии для их выбора. Рассмотрим каждый из этих способов по очереди. В этом разделе мы будем использовать окончательную, очищенную версию объектов в файле `data/geovarseas.csv`.

6.1 Создание новых объектов, например, административных средних

Мы будем оценивать модели на уровне `admin4`. Однако это не означает, что все наши предикторы обязательно должны быть на уровне `admin4`. Например, мы можем создать средние или стандартные отклонения на более высоких уровнях (напр., `admin3`). Мы также можем создать запаздывающие средние или стандартные отклонения на уровне `admin4`, а также взаимодействие между различными объектами.

В качестве простого примера рассмотрим создание переменной для общей численности населения на уровне `admin3` в дополнение к уровню `admin4`. Давайте сначала загрузим объекты в среду R и немного почистим их:

```
1 # загрузка элементов
2 features <- read_csv("data/geovarseas.csv")
3 # столбец с данными о численности населения называется mwpop
4 head(features)

# A tibble: 6 x 526
  EA_CODE TA_CODE mwpop average_masked barecoverfraction urbancoverfraction
  <dbl>   <dbl> <dbl>         <dbl>         <dbl>         <dbl>
1 10507801 10507   700.         0.585             407             769
2 10507072 10507   940.           0             904             114
3 10507010 10507  1077.           0            1525            3101
4 10507001 10507   610.           0             673             1919
5 10507009 10507   558.           0            1383             957
6 10507033 10507   594.           0             738            1802
# i 520 more variables: cropscoverfraction <dbl>, grasscoverfraction <dbl>,
# mosscoverfraction <dbl>, waterpermanentcoverfraction <dbl>,
# waterseasonalcoverfraction <dbl>, shrubcoverfraction <dbl>,
# snowcoverfraction <dbl>, treecoverfraction <dbl>, ndvi1 <dbl>, ndvi2 <dbl>,
# ndvi3 <dbl>, ndvi4 <dbl>, ndvi5 <dbl>, ndvi6 <dbl>, ndvi7 <dbl>,
# ndvi8 <dbl>, ndvi9 <dbl>, ndvi10 <dbl>, ndvi11 <dbl>, ndvi12 <dbl>,
# mosaik1 <dbl>, mosaik2 <dbl>, mosaik3 <dbl>, mosaik4 <dbl>, ...
```

Теперь создадим еще одну переменную для общей численности населения на уровне `admin3`:

```

1 # создаем переменную для общей численности населения на уровне admin3:
2 features <- features |>
3   group_by(TA_CODE) |>
4   mutate(popTA = sum(mwpop, na.rm = TRUE)) |>
5   ungroup()

```

Здесь мы используем возможности функций `group_by()` и `mutate()` пакета `tidyverse` для агрегирования общей численности населения до уровня `admin3`, но сохраняем сам набор данных на уровне `admin4`.

В качестве другого примера рассмотрим индекс NDVI, который является показателем растительности («зелености»). В таких странах, как Малави, индекс NDVI может в значительной степени предсказывать уровень бедности, поскольку он коррелирует с урожаем сельскохозяйственных культур. Однако в то же время точное время измерения индекса NDVI может быть очень важным для улавливания этой взаимосвязи в силу сезонности, присущей неорошаемому сельскохозяйственному производству. На практике мы зачастую берем исторические данные индекса NDVI, а затем добавляем к текущим значениям такие переменные, как долгосрочное среднее, долгосрочный максимум, долгосрочный минимум и даже долгосрочное стандартное отклонение.

В нашем простом примере мы скачали 12 отдельных файлов с данными NDVI²⁹ и можем использовать их для расчета таких переменных для NDVI, как `min/mean/sd` (минимум/среднее/стандартное отклонение) в течение года. В объекте `features` столбцы с индексом NDVI названы `ndviM`, где `M` — целое число от 1 до 12, указывающее на месяц года.

Давайте создадим несколько новых значений, таких как *годовые* среднее, максимальное, минимальное и стандартное отклонение:

```

1 # Сначала найдем нужные нам столбцы!
2 ndvicols <- grep("ndvi", names(features))
3 features$ndvimean <- apply(features[,ndvicols], 1, mean, na.rm = TRUE)
4 features$ndvistd <- apply(features[,ndvicols], 1, sd, na.rm = TRUE)
5 features$ndvimax <- apply(features[,ndvicols], 1, max, na.rm = TRUE)
6 features$ndvimin <- apply(features[,ndvicols], 1, min, na.rm = TRUE)
7 head(features[,c("ndvimean", "ndvistd", "ndvimax", "ndvimin")])

```

```

# A tibble: 6 x 4
  ndvimean ndvistd ndvimax ndvimin
    <dbl>    <dbl>    <dbl>    <dbl>
1   9066.    2787.    13106     5681
2  23035.    8614.    36599    13413
3  24912.    8605.    37987    15263
4   31705    8246.    42600    20608
5   23144.    7180.    33499    14993
6   13229.    3927.    19218     8390

```

Здесь мы используем новую функцию `apply()`, которую ранее не применяли. Она очень полезна. В данном случае мы собираемся «применить» одну функцию к разным столбцам объекта `ndviextracted`. Пройдемся по каждой строке в приведенном выше коде:

²⁹Это цикл `for` в конце блокнота `geospatialpull.ipynb`.

1. Эта строка выполняет несколько более сложную задачу. Мы хотим найти расположение столбцов, которые содержат строку «ndvi» в названиях столбцов. Мы можем сделать это с помощью функции `grep()`, которая возвращает *индекс* столбцов, содержащих строку «ndvi». Затем мы можем использовать этот индекс для поиска нужных нам столбцов.
2. `ndvimean`: Мы собираемся взять среднее значение NDVI для каждой *строки*, что в вызове функции обозначается цифрой 1 (если бы мы хотели применить ее *вниз* к столбцам, мы бы использовали цифру 2). С помощью функции `apply()` применяем функцию `mean()` к каждой строке, за исключением тех, в которых отсутствуют значения. Следующие три строки делают нечто подобное, только вместо функции `mean` вызывается другая функция.
3. Здесь мы смотрим на первые несколько строк объекта `df`, но *только* применительно к новым столбцам, которые мы только что создали.

6.2 Размышления о преобразованиях

Помимо создания новых переменных путем агрегирования на более высоком географическом уровне или объединения данных во времени, мы также можем преобразовать как наши (потенциальные) переменные-предикторы, так и переменные-результаты, например, с помощью логарифмических преобразований. Почему мы можем захотеть преобразовать наши переменные? Есть две основные причины:

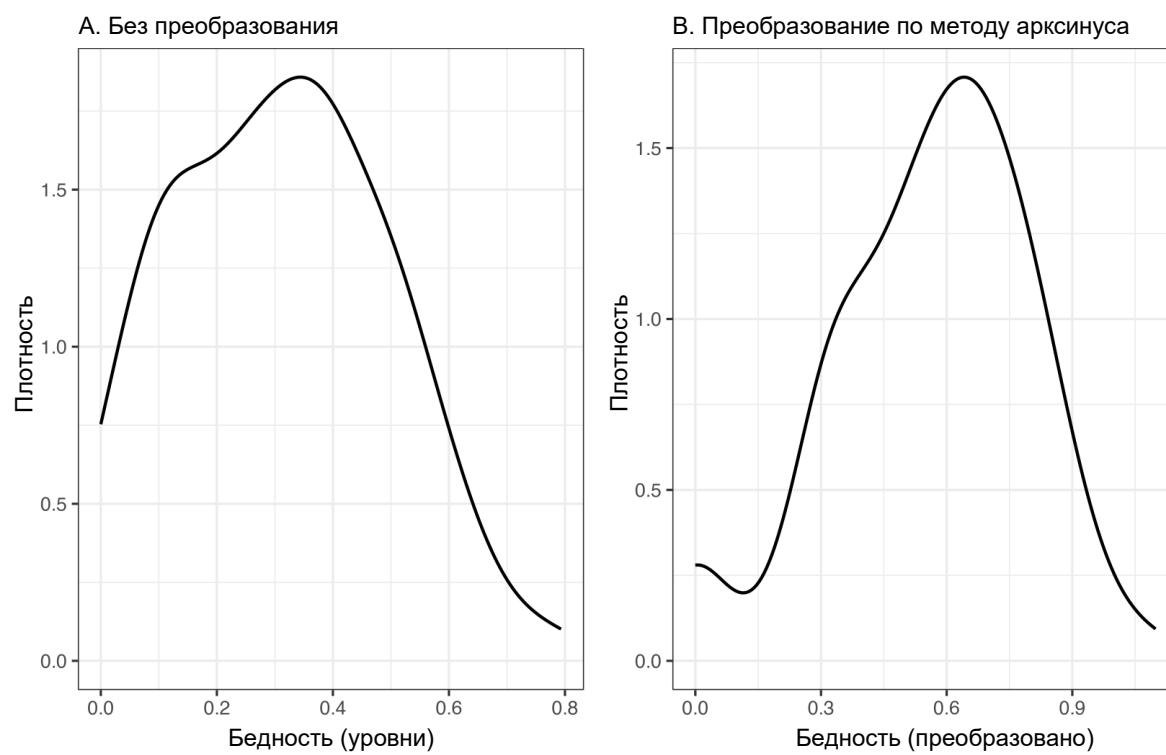
1. Повышение предсказательной способности модели. Например, логарифмическое преобразование иногда может сделать связь между предиктором и результатом более линейной, что особенно важно для линейных моделей (которые являются наиболее надежными для ОМР).
2. Улучшение свойств остаточных эффектов. При проведении ОМР мы нередко применяем параметрический бутстреп-метод для точечных оценок и выводов. В нашем случае мы делаем два ключевых предположения: остаточные эффекты распределены нормально и случайные эффекты распределены нормально. Преобразование переменной результата, в частности, иногда может нормализовать остаточные эффекты, что может улучшить свойства бутстреп-метода.

Начнем с результата – показателей бедности на уровне `admin4`. Прежде чем перейти к их разбору, важно пояснить, что предположения, которые мы делаем в нашей модели ОМР, касаются не распределения самого результата, а скорее *остаточных эффектов* и случайных эффектов. Тем не менее, результаты с более «нормальным» распределением, как правило, имеют лучшие свойства с точки зрения оценки.

На рисунке ниже показана плотность уровней бедности в Северной провинции Малави. Слева показаны уровни бедности без преобразования, а справа – уровни бедности после преобразования по методу арксинуса (квадратного корня)³⁰.

³⁰Преобразование по методу арксинуса (квадратного корня) определяется как $y^* = \sin^{-1}(\sqrt{y})$.

Рисунок 13: Уровни бедности в Северной провинции Малави



Мы обнаружили, что это преобразование работает особенно хорошо, когда результат представляет собой пропорцию, как в данном случае (она варьируется между 0 и 1). На рисунке хорошо видно, что преобразованный результат более «нормальный», чем непреобразованный. Однако, поскольку нас действительно интересуют остаточные эффекты, ниже мы подробнее обсудим диагностику модели, где также рассмотрим статистику показателей асимметрии и эксцесса остаточных и случайных эффектов.

Так как нас действительно интересуют остаточные эффекты, мы можем посмотреть, как преобразование может повлиять на прогностическую способность модели. Для этого мы воспользуемся функцией `feols()` из пакета `fixest`³¹.

Однако перед этим нам нужно перекодировать некоторые из ключевых предикторов, относящихся к классификации почвенно-растительного покрова. На данный момент значения классификации почвенно-растительного покрова не являются истинными пропорциями! Они представляют собой количество пикселей, относящихся к разным категориям земель в пределах каждого СУ. Нам нужно преобразовать их в пропорции. Это можно сделать, разделив на общее количество пикселей в каждом СУ. Что мы и сделаем:

```

1 # найдем нужные нам столбцы
2 landcols <- grep("coverfraction", names(features))
3 # сколько всего пикселей?
4 features$totalpixels <- apply(features[,landcols], 1, sum, na.rm = TRUE)
5 # пройдемся по каждому и заменим на пропорцию:
6 for (i in landcols){
7   features[,i] <- features[,i]/features$totalpixels
8 }
9 # удалим общее количество пикселей
10 features <- features |>
11   select(-totalpixels)
12 summary(features[,landcols])

```

barecoverfraction	urbancoverfraction	cropscoverfraction	grasscoverfraction
Min. :0.000000	Min. :0.000000	Min. :0.0000	Min. :0.0000
1st Qu.:0.005634	1st Qu.:0.009176	1st Qu.:0.2756	1st Qu.:0.2449
Median :0.012542	Median :0.021913	Median :0.3707	Median :0.2660
Mean :0.014802	Mean :0.103282	Mean :0.3428	Mean :0.2533
3rd Qu.:0.021850	3rd Qu.:0.051555	3rd Qu.:0.4351	3rd Qu.:0.2904
Max. :0.091131	Max. :1.000000	Max. :0.6276	Max. :0.5254
mosscoverfraction	waterpermanentcoverfraction	waterseasonalcoverfraction	
Min. :0	Min. :0.0000000	Min. :0.000000	
1st Qu.:0	1st Qu.:0.0000000	1st Qu.:0.000000	
Median :0	Median :0.0000000	Median :0.000000	
Mean :0	Mean :0.0008774	Mean :0.001152	
3rd Qu.:0	3rd Qu.:0.0000000	3rd Qu.:0.000000	
Max. :0	Max. :0.1184433	Max. :0.161355	
shrubcoverfraction	snowcoverfraction	treecoverfraction	
Min. :0.0000	Min. :0	Min. :0.00000	
1st Qu.:0.1160	1st Qu.:0	1st Qu.:0.08103	

³¹ Также вы можете использовать функцию `lm()` из базового функционала языка R. Мы же предпочитаем функцию `feols()`, поскольку она значительно упрощает некоторые задачи, например, добавление весов.

Median :0.1397	Median :0	Median :0.11777
Mean :0.1307	Mean :0	Mean :0.15302
3rd Qu.:0.1594	3rd Qu.:0	3rd Qu.:0.19193
Max. :0.2394	Max. :0	Max. :0.87709

Приведенные выше результаты также дают некоторый дополнительный контекст для выбора переменных для наших моделей ОМР. В нашем примере ОМР, описанном ниже, мы будем оценивать модель с помощью оценки максимального правдоподобия. В таких случаях иногда возникают проблемы с конвергенцией, особенно если некоторые предикторы имеют очень малую вариацию. В данном примере некоторые переменные классификации растительного покрова не имеют вариации (поэтому они вообще не будут выбраны при применении метода лассо-регрессии, который мы рассмотрим ниже), а другие показывают очень малую вариацию. Давайте удалим все столбцы с очень малой вариацией:

```
1 features <- features |>
2   select(-c("mosscoverfraction", "waterpermanentcoverfraction",
3     "waterseasonalcoverfraction", "snowcoverfraction"))
```

Рассмотрим все три регрессии: слева – бедность, справа – два отдельных предиктора: `cropscoverfraction` и `mwpop`. Мы оценим три отдельные (простые) регрессии с различными преобразованиями. В первом столбце все переменные, в т.ч. результат, не преобразованы. Во втором столбце мы преобразуем только результат, оставляя оба предиктора без преобразования. В третьем столбце мы преобразуем и результат, и предикторы, используя преобразование по методу арксинуса для площади культур и преобразование по методу логарифма для численности населения. Результаты представлены в Таблице 1.

```
1 # новые данные
2 pov <- read_csv("data/ihs5ea.csv")
3 # теперь объединяем pov с объектами. Это означает, что у нас есть все районы
  ↳ уровня admin4, будь то с данными выборки или без них
4 pov <- features |>
5   left_join(pov, by = "EA_CODE") |>
6   mutate(mwpop = mwpop/1000)
7
8 reg1 <- feols(poor ~ cropscoverfraction + mwpop, data = pov, weights =
  ↳ ~total_weights, vcov = "HC1")
9 reg2 <- feols(asin(sqrt(poor)) ~ cropscoverfraction + mwpop, data = pov,
  ↳ weights = ~total_weights, vcov = "HC1")
10 reg3 <- feols(asin(sqrt(poor)) ~ asin(sqrt(cropscoverfraction)) + log(mwpop),
  ↳ data = pov, weights = ~total_weights, vcov = "HC1")
```

Преобразование только результата улучшает подгонку модели, по крайней мере, по коэффициенту детерминации R-квадрат, приблизительно на 7,2 процента. При этом все преобразования увеличивают предсказательную способность регрессии на 27,8 процента.

Таблица 1: Преобразование переменных

	Бедность (Poor)	Преобразованные	
		Бедность (Poor)	Бедность (Poor)
Площадь культур (уровни)	0,4091 (0,1468)	0,5272 (0,1767)	
Население (уровни, 2000-е)	-0,1185 (0,0555)	-0,1468 (0,0635)	
Площадь культур (арксинус)			0,4911 (0,1421)
Население (логарифм)			-0,1272 (0,0403)
Константа	0,2950 (0,0535)	0,5405 (0,0627)	0,2584 (0,0969)
Наблюдения	107	107	107
R ²	0,097	0,104	0,124

Примечание: В таблице представлены результаты регрессии переменных бедности на перечисленные переменные. В первом столбце итоговая переменная (бедность) не преобразована. Во втором и третьем столбцах итоговая переменная преобразована с помощью метода арксинуса (квадратного корня). К предикторам для показателей площади культур и населения применены преобразования с помощью метода арксинуса и логарифма, соответственно. В скобках приведены стандартные ошибки с поправкой на гетероскедастичность.

Другими словами, преобразования только этих двух переменных, а также результата могут привести к значительному улучшению модели³².

6.3 Лассо-регрессия и пакет `glmnet`

В предыдущих разделах мы показали, как преобразование переменных-предикторов и переменных-результатов может улучшить подгонку модели. Однако мы еще не обсудили, как выбрать, какие именно объекты следует включить в модель. Включение всех потенциальных предикторов сопряжено с двумя основными проблемами:

1. Во-первых, нередко предикторов бывает больше, чем наблюдений. В этом случае модель оценить невозможно.
2. Даже если наблюдений у нас больше, чем предикторов, включение всех предикторов может привести к чрезмерной подгонке. По сути, в итоге наша модель может предсказывать «помехи» в данных, а не истинные базовые взаимосвязи. Это приведет к очень низким по качеству результатам, особенно при прогнозировании на основе данных, не входящих в выборку.

Существует множество способов отбора предикторов, но наиболее часто используется метод лассо-регрессии³³. Лассо – это метод регрессии, который «штрафует» за большие значения коэффициентов. Проще говоря, он уменьшает коэффициенты до нуля, если они по существу не улучшают работу модели. На практике это приблизительно уравнивает коэффициент детерминации R-квадрат в выборке и за ее пределами.

В своей простейшей форме метод лассо-регрессии представляет собой линейную регрессионную модель с дополнительным «штрафным» членом. Мы минимизируем следующую целевую функцию в отношении β :

$$(y - X\beta)^2 + \lambda \sum_{j=1}^p |\beta_j|,$$

где λ – параметр настройки, определяющий размер «штрафа», а $(y - X\beta)^2$ – обычная задача минимизации по методу наименьших квадратов. Важно отметить, что штрафной член в принципе может принимать любое (неотрицательное) значение. Когда $\lambda = 0$, лассо является простой линейной регрессией. По мере увеличения λ будет выбираться все меньше и меньше переменных (т.е. с ненулевыми коэффициентами). Так каково же «правильное» значение λ ? На практике мы часто выбираем значение λ с помощью метода перекрестной проверки.

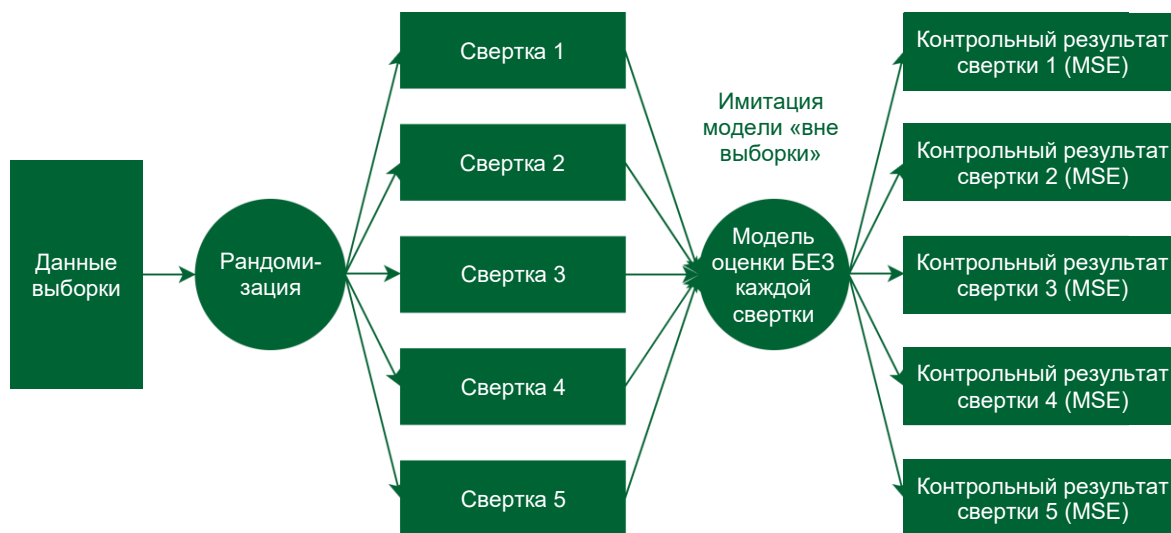
Перекрестная проверка – это процесс, призванный имитировать оценку за пределами выборки. Рассмотрим схему на рис. 14. Сначала мы берем данные выборки и случайным

³² Отметим, что мы приводим эти результаты только в целях наглядности. На практике мы не используем коэффициент детерминации R-квадрат в качестве меры подгонки модели, особенно если смотреть только в пределах выборки.

³³ LASSO расшифровывается как «least absolute shrinkage and selection operator» (оператор наименьшего абсолютного сокращения и выбора), но его зачастую обозначают просто аббревиатурой.

образом разбиваем их на N частей (*сверток*); наиболее распространенное количество сверток – 10, но для простоты на диаграмме показано пять. Поскольку мы пытаемся имитировать работу модели за пределами выборки, мы оцениваем модель на $N - 1$ свертках, а затем прогнозируем на оставшихся свертках. Затем мы вычисляем среднеквадратичную ошибку (MSE) прогноза. Мы повторяем этот процесс N раз, каждый раз исключая разные свертки. Затем мы можем найти среднее значение MSE по всем сверткам. Мы можем повторять этот процесс много раз для разных значений λ . Окончательным («оптимальным») значением λ будет то, которое минимизирует среднее значение MSE по всем сверткам.

Рисунок 14: Схема перекрестной проверки



К счастью, нам не нужно делать все это вручную. Вместо этого мы можем воспользоваться функцией `cv.glmnet()` из пакета `glmnet`. Она автоматически распределит наблюдения по сверткам и вычислит MSE для различных значений λ . Для этого необходимо установить пакет `glmnet`, что можно сделать с помощью функции `install.packages("glmnet")`, а затем загрузить библиотеку с помощью функции `library(glmnet)`.

Следующий код иллюстрирует этот процесс на примере уже очищенных данных. Очищенные объекты находятся в файле `geovars.csv`, а результат – в файле `ihs5ea.csv`. Оба набора данных относятся к Северной провинции Малави и уже свернуты до уровня `admin4` (СУ). Сначала загрузим оба набора данных:

```

1 # загрузим данные по бедности
2 pov <- read_csv("data/ihs5ea.csv")
3 # загрузим объекты
4 features <- read_csv("data/geovarseas.csv")
5 # добавим объекты к данным по бедности
6 pov <- pov |>
7   left_join(features, by = "EA_CODE")
8 head(pov)

```

```
# A tibble: 6 x 530
  EA_CODE   poor total_weights total_obs fold TA_CODE mwpop average_masked
  <dbl>   <dbl>         <dbl>    <dbl> <dbl>   <dbl>   <dbl>         <dbl>
1 10101006 0.230           5690.      16     4   10101   1123.         0.519
2 10101011 0.444           7614.      16     2   10101   1224. 0
3 10101027 0.0947          9441.      16     5   10101   1292. 4.27
4 10101033 0.376           7486.      16     4   10101    672. 0
5 10101039 0.600           9147.      16     1   10101   1056. 0
6 10101054 0.497           5351.      16     5   10101    864. 0
# i 522 more variables: barecoverfraction <dbl>, urbancoverfraction <dbl>,
#   cropscoverfraction <dbl>, grasscoverfraction <dbl>,
#   mosscoverfraction <dbl>, waterpermanentcoverfraction <dbl>,
#   waterseasonalcoverfraction <dbl>, shrubcoverfraction <dbl>,
#   snowcoverfraction <dbl>, treecoverfraction <dbl>, ndvi1 <dbl>, ndvi2 <dbl>,
#   ndvi3 <dbl>, ndvi4 <dbl>, ndvi5 <dbl>, ndvi6 <dbl>, ndvi7 <dbl>,
#   ndvi8 <dbl>, ndvi9 <dbl>, ndvi10 <dbl>, ndvi11 <dbl>, ndvi12 <dbl>, ...
```

Мы должны знать, в каком столбце находится интересующий нас результат (в данном случае – `poor`), а в каких столбцах – все предикторы. Данные `geovarseas.csv` были очищены таким образом, что в них есть только две переменные, не являющиеся предикторами: идентификатор `admin4` (`EA_CODE`) и идентификатор `admin3` (`TA_CODE`). Это означает, что все столбцы от `mwpop` до `ncol(pov)` являются предикторами. Прежде чем использовать функцию `cv.glmnet`, мы должны четко определить, чем являются `Y` и `X` в данном случае³⁴. Кроме того, вспомните, что мы обсуждали, в частности, использование преобразования переменной результата. Давайте сделаем это и здесь:

```
1 Y <- as.vector(asin(sqrt(pov$poor)))
2 # данные mwpop находятся в столбце 6
3 X <- as.matrix(pov[,7:ncol(pov)])
4 # вот перекрестная проверка для выбора лямбды
5 set.seed(234056) # это нужно для обеспечения последовательности результатов!
6 # пять сверток для упрощения примера
7 cvresults <- cv.glmnet(x = X, y = Y, nfolds = 5)
8 cvresults
```

Call: `cv.glmnet(x = X, y = Y, nfolds = 5)`

Measure: Mean-Squared Error

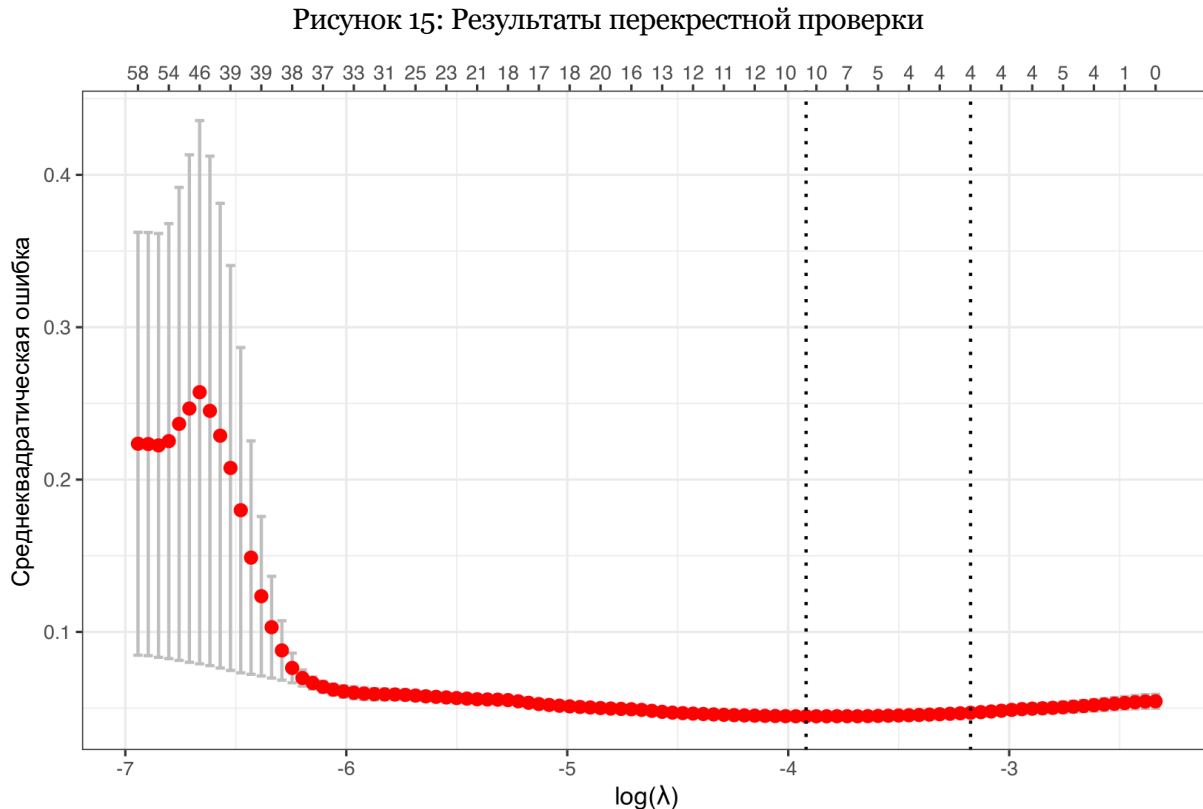
	Lambda	Index	Measure	SE	Nonzero
min	0.01986	35	0.04466	0.002446	10
1se	0.04180	19	0.04697	0.002626	4

Объект `cvresults` содержит информацию о двух разных вариантах значений λ : значение λ , которое минимизирует MSE (`lambda.min`), и значение λ , которое чуть больше (`lambda.1se`), то есть оно более консервативно и обычно приводит к меньшему количеству ненулевых коэффициентов. В данном примере число ненулевых коэффициентов равно

³⁴ Небольшое примечание: результат должен быть вектором, а предикторы должны быть в виде матрицы. Оба эти требования указаны в коде.

либо 10, либо 4, в зависимости от того, какое значение λ мы выберем³⁵. С помощью функции `plot()` мы можем посмотреть на результат, который представлен на рис. 15 (пунктирными линиями показаны два «оптимальных» значения λ).

```
1 plot(cvresults)
```



Следующий шаг является самым важным – извлечение переменных с ненулевыми коэффициентами. Если с коэффициентами все просто, то извлечение имен переменных оказывается гораздо более сложной задачей, чем это, вероятно, должно быть.

Чтобы просто посмотреть коэффициенты, мы можем использовать функцию `coef()`, как показано ниже. Обратите внимание, что в коде используется функция `head()`, чтобы размер вывода на языке R был приемлемым для данного руководства³⁶:

```
1 head(coef(cvresults))
```

³⁵ Обратите внимание, что нам необходимо задать начальное значение (`set.seed()`), если мы хотим получить последовательные результаты при повторном выполнении перекрестной проверки. Это связано с тем, что перекрестная проверка опирается на некоторую степень случайности (в распределении по сверткам).

³⁶ Двумя неотрицательными переменными, которые мы можем видеть в выводе, являются уровень ночной освещенности (`average_masked`) и процент территории без растительности (`barecoverfraction`). Оба эти коэффициента имеют ожидаемое направление.

```
6 x 1 sparse Matrix of class "dgCMatrix"
              s1
(Intercept)    5.752724e-01
mwpop          .
average_masked -1.427754e-02
barecoverfraction 3.923403e-05
urbancoverfraction .
cropscoverfraction .
```

Коэффициенты в виде точки имеют нулевое значение. Мы можем использовать этот факт, чтобы извлечь имена переменных с ненулевыми коэффициентами. В этом можно убедиться, если преобразовать результат `coef()` в матрицу³⁷:

```
1 head(as.matrix(coef(cvresults)))
```

```
              s1
(Intercept)    5.752724e-01
mwpop          0.000000e+00
average_masked -1.427754e-02
barecoverfraction 3.923403e-05
urbancoverfraction 0.000000e+00
cropscoverfraction 0.000000e+00
```

Преобразовав результаты в матрицу, мы видим, что коэффициенты в виде точки стали нулями. Теперь мы можем извлечь имена ненулевых коэффициентов следующим образом:

```
1 # мы можем указать лямбду, которую хотим использовать; "lambda.min" или "lambda.1se"
2 nonzero <- as.matrix(coef(cvresults, "lambda.min"))
3 nonzero <- rownames(nonzero)[nonzero[,1]!=0]
4 nonzero
```

```
[1] "(Intercept)"      "average_masked"    "barecoverfraction"
[4] "urbancoverfraction" "mosaik1"           "mosaik39"
[7] "mosaik156"         "mosaik268"         "mosaik396"
[10] "mosaik459"         "mosaik464"
```

```
1 # теперь удалим intercept (потому что он добавляется автоматически)
2 nonzero <- nonzero[-1]
3 nonzero
```

```
[1] "average_masked"      "barecoverfraction"  "urbancoverfraction"
[4] "mosaik1"             "mosaik39"          "mosaik156"
[7] "mosaik268"           "mosaik396"         "mosaik459"
[10] "mosaik464"
```

³⁷ Снова используем функцию `head()`, чтобы ограничить вывод первыми несколькими переменными для практических целей.

В этом примере у нас есть три ненулевых коэффициента, но мы хотим удалить свободный коэффициент (intercept), который всегда является первым коэффициентом. Мы удаляем его, потому что он будет автоматически добавлен функцией OMP, которую мы используем позже. Это делается с помощью строки кода `nonzero <- nonzero[-1]` выше.

Последний шаг – превратить это в *формулу* вида *outcome* $x_1 + \dots + x_n$, где $x_1, \dots, x_n$ – ненулевые коэффициенты. Мы можем сделать это с помощью функции `paste()` и опции `collapse` следующим образом:

```
1 ebpformula <- as.formula(paste("poor ~ ", paste(nonzero, collapse = " + ")))
2 ebpformula
```

```
poor ~ average_masked + barecoverfraction + urbancoverfraction +
      mosaik1 + mosaik39 + mosaik156 + mosaik268 + mosaik396 +
      mosaik459 + mosaik464
```

Опция `collapse = " + "` сообщает функции `paste()`, что нужно разделить переменные знаком плюса. И наконец, мы должны явно указать языку R, что это формула, что мы и делаем с помощью функции `as.formula()`. Эту формулу мы будем использовать ниже в разделе [7](#).

7 Оценка модели

В этом разделе мы оценим модель, которую мы строили до этого. Мы будем использовать обновленную версию пакета `povmap`, которого еще нет на ресурсе CRAN, поэтому мы скачаем его с GitHub.

7.1 Пакет `povmap`

Пакет `povmap` содержит множество функций, позволяющих оценивать малые районы различными методами. Пакет находится в активной разработке, но его версия на ресурсе CRAN является стабильной. Вы можете найти pdf-документацию к пакету [здесь](#), а более обновленные версии, которых еще нет на ресурсе CRAN, можно найти на странице GitHub Статистической группы Всемирного банка для стран Африки к югу от Сахары [здесь](#). Как уже отмечалось, мы не будем использовать версию, доступную на ресурсе CRAN. Вместо этого мы будем использовать более обновленную версию, которая включает различные опции для функции `ebp()`. Вот код для установки пакета с GitHub:

```
1 devtools::install_github("SSA-Statistical-Team-Projects/povmap", ref = "david3")
```

На тот случай, если вы получите ошибку при попытке установить пакет `povmap`, например, если авторы пакета обновят версию на ресурсе CRAN и удалят ветку `david3`, мы также выложили его `zip`-файл на нашей [странице GitHub](#). Можете скачать его и установить с помощью команды `devtools::install_local(PATH)`, где `PATH` – путь к `zip`-файлу.

В этом примере мы будем использовать функцию `ebp()`, которая обозначает наилучший эмпирический прогноз ([Molina and Rao 2010](#)). Мы будем оценивать модель субрайона, в которой географический уровень `admin4` является субрайоном, а прогнозирование осуществляется на уровне `admin3` (т.е. ТВ – или традиционные власти – в Малави).

В предыдущем разделе мы уже подготовили наши данные для оценки. Наша переменная результата содержится в объекте `pov`, а наши предикторы – в объекте `features`.

Простейшая формулировка нашей модели выглядит следующим образом:

```
1 results <- ebp(  
2   fixed = ebpformula, ①  
3   pop_data = features, ②  
4   pop_domains = "TA_CODE", ③  
5   smp_data = pov, ④  
6   smp_domains = "TA_CODE", ⑤  
7   MSE = TRUE, ⑥  
8   transformation = "arcsin", ⑦  
9   na.rm = TRUE ⑧  
10 )
```

- ① `fixed = ebpformula`: Это формула, которую мы создали выше и которая включает ненулевые коэффициенты, отобранные с помощью метода лассо-регрессии.

- ② `pop_data = features`: Это наш набор данных, содержащий предикторы для *численности населения*. В данном случае это объект `features`.
- ③ `pop_domains = "TA_CODE"`: Идентификатор района в данных о численности населения. В нашем примере это идентификатор уровня `admin3` – `TA_CODE`.
- ④ `smp_data = pov`: Набор данных, содержащий интересующий нас результат. Он создан на основе опроса, поэтому не охватывает все районы уровня `admin3`. В нашем примере это объект `pov`.
- ⑤ `smp_domains = "TA_CODE"`: Идентификатор района в данных выборки. В данном случае он имеет то же имя, что и объект `pop_domains`.
- ⑥ `MSE = TRUE`: Этот код сообщает функции, что нужно вычислить оценку дисперсии³⁸.
- ⑦ `transformation = "arcsin"`: Это преобразование, которое мы использовали выше. Обратите внимание, что по умолчанию функция `ebp()` выполняет преобразование Бокса-Кокса. Если вы не хотите выполнять преобразование, можете указать `transformation = "no"`. Важно отметить, что нам нужно передать функции `ebp()` *нетрансформированную* переменную результата в объекте `pov`, поскольку позже для преобразования мы используем функцию `ebp90`.
- ⑧ `na.rm = TRUE`: Это сообщает функции, что нужно удалить отсутствующие значения. Это важно, потому что функция не будет работать при наличии отсутствующих значений. В идеале нужно очистить данные так, чтобы эта опция не была строго необходима (т.е. отсутствующих значений уже не будет).

7.2 Определение таких опций, как взвешивание, преобразование, задание эталонных значений

Мы уже видели, как используется одна из опций – преобразование. Но функция `ebp()` принимает и другие опции. Давайте обсудим в частности опции взвешивания и задания эталонных значений.

Для взвешивания можно указать два разных типа весовых коэффициентов: выборочные веса и веса численности населения.

- Выборочные веса – это такие веса, которые используются для поправки на то, что вероятность попадания одних наблюдений в выборку может быть выше, чем других. В нашем случае мы уже создали эти веса в объекте `pov`; они находятся в столбце под названием `total_weights`.
- Веса численности населения используются для поправки на то, что в разных подрайонах проживает разное количество людей. Поскольку мы хотим рассчитать взвешенный по численности населения уровень бедности на уровне `admin3`, мы можем добавить опцию `pop_weights`. Расчетная численность населения (по данным WorldPop) содержится в столбце `mwpop` объекта `features`.

³⁸Важно отметить, что мы оцениваем среднюю квадратичную ошибку (MSE), а не дисперсию. Однако мы обычно предполагаем, что они одинаковы, исходя из предположения об отсутствии ошибок случайной выборки.

Мы также можем *задать эталонные значения* оценкам в соответствии с существующими официальными статистическими данными для более высоких географических уровней. В нашем примере мы располагаем данными только по Северной провинции Малави. В Отчете о бедности в Малави за 2020 год ([2020 Malawi Poverty Report](#)) говорится, что официальный уровень бедности в этой провинции составляет 32,9 процента. Мы используем пропорции, поэтому хотим задать общие эталонные значения таким образом, чтобы они в среднем равнялись 0,329. Нам нужно передать функции `ebp` *именованный вектор*, где имя равно показателю, для которого мы хотим задать эталонное значение, а значение равно эталонному значению. Функция может задавать эталонные значения для векторов "Mean" или "Head_Count". Хотя мы оцениваем бедность по численности населения, на самом деле мы оцениваем средний уровень бедности в каждом районе уровня `admin3`. Мы можем создать именованный вектор следующим образом:

```
1 bench <- c("Mean" = 0.329)
```

Собрав все это вместе, мы можем оценить нашу окончательную модель:

```
1 results <- ebp(  
2   fixed = ebpformula,  
3   pop_data = features,  
4   pop_domains = "TA_CODE",  
5   smp_data = pov,  
6   smp_domains = "TA_CODE",  
7   MSE = TRUE,  
8   transformation = "arcsin",  
9   na.rm = TRUE,  
10  weights = "total_weights",  
11  pop_weights = "mwpop",  
12  weights_type = "nlme",  
13  benchmark = bench  
14 )
```

- ① `weights = "total_weights"`: Это выборочный вес. Он используется для поправки на то, что данные обследования не являются простой случайной выборкой.
- ② `pop_weights = "mwpop"`: Весовой коэффициент численности населения. Используется для поправки на то, что в разных подрайонах проживает разное количество людей.
- ③ `weights_type = "nlme"`: Эта опция необходима при использовании преобразования по методу арксинуса. К сожалению, тип взвешивания по умолчанию не работает с преобразованием по методу арксинуса.
- ④ `benchmark = bench`: Это опция задания эталонных значений. Здесь мы устанавливаем средний уровень бедности в среднем равным 0,329.

7.3 Проверка предположений

Теперь, когда мы оценили модель, мы можем посмотреть на некоторые статистические данные, чтобы проверить предположения нашей модели. Поскольку мы используем параметрический бутстреп-метод, предположение о нормальности здесь важно как для

остаточных эффектов, так и для случайных эффектов. Мы можем посмотреть статистику нормальности с помощью функции `summary()`:

```
1 summary(results)
```

Empirical Best Prediction

Call:

```
ebp(fixed = poor ~ average_masked + barecoverfraction + urbancoverfraction +
  mosaik1 + mosaik39 + mosaik156 + mosaik268 + mosaik396 +
  mosaik459 + mosaik464, pop_data = features, pop_domains = "TA_CODE",
  smp_data = pov, smp_domains = "TA_CODE", transformation = "arcsin",
  MSE = TRUE, na.rm = TRUE, weights = "total_weights", pop_weights = "mwpop",
  weights_type = "nlme", benchmark = bench)
```

Out-of-sample domains: 27
 In-sample domains: 49
 Out-of-sample subdomains: 0
 In-sample subdomains: 0

Sample sizes:
 Units in sample: 107
 Units in population: 2908

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
Sample_domains	1	1.0	2	2.183673	3	7
Population_domains	1	5.5	18	38.263158	44	300

Explanatory measures for the mixed model:

Marginal_R2	Conditional_R2	Marginal_Area_R2	Conditional_Area_R2
0.3064676	0.363829	0.6806806	0.7700488

Residual diagnostics for the mixed model:

	Skewness	Kurtosis	Shapiro_W	Shapiro_p
Error	-0.1317611	3.652988	0.9909934	0.70316526
Random_effect	0.3867831	5.282390	0.9465488	0.02682613

Estimated variance of random effects:

	Variance
Error	0.030851330
Random_effect	0.002781762

ICC: 0.08270908

Shrinkage factors

	gamma.Min.	gamma.1st.Qu.	gamma.Median	gamma.Mean	gamma.3rd.Qu.
TA_CODE	0.08270908	0.08270908	0.1416945	0.1477909	0.189778
	gamma.Max.				
TA_CODE	0.378302				

Transformation:

Transformation	Shift_parameter
arcsin	0

В частности, в выходных данных есть раздел «Residual diagnostics for the mixed model» (Диагностика остаточных эффектов для смешанной модели). Здесь мы видим, что у нас присутствуют асимметрия и эксцесс. Для нормального распределения они должны быть равны нулю и трем, соответственно. В данном случае эксцесс немного высок как для остаточных эффектов, так и для случайных эффектов. Это не идеально, но, учитывая их значения, это также не повод для беспокойства. Мы также можем посмотреть результаты, используя функцию `plot(results)`³⁹:

```
1 plot(results)
```

7.4 Оценка результатов

Последний шаг – оценка результатов. Мы можем сделать это несколькими способами. Во-первых, мы можем посмотреть на значение коэффициента детерминации R-квадрат. Во-вторых, мы можем посмотреть на изменение точности результатов от выборочных до смоделированных оценок. В-третьих, мы можем проверить точность оценок. Обсудим каждый из этих способов по очереди.

7.4.1 Коэффициент детерминации (R-квадрат)

Функция `summary()` выше также выводит информацию о коэффициенте детерминации. Существует четыре отдельных значения этого коэффициента: R-квадрат маргинальных районов, R-квадрат маргинальных единиц, R-квадрат условных районов и R-квадрат условных единиц. По опыту, именно значения R-квадрата районов наиболее значимы для качества результатов. Это объясняет долю дисперсии по *районам*, которая объяснена моделью (либо одними только коэффициентами, либо и коэффициентами, и случайными эффектами). В нашем примере R-квадрат условных районов составляет примерно 0,77, что является относительно высоким показателем.

7.4.2 Как сильно меняется точность?

Давайте сравним новые оценки с оценками выборки. Сначала извлечем результаты из объекта `results`. Нам нужно извлечь три вещи: идентификатор `admin3`, смоделированный уровень бедности и смоделированные оценки дисперсии:

```
1 resultsebp <- as_tibble(cbind(TA_CODE = levels(results$ind$Domain),  
2   poor_ebp = results$ind$Mean_bench,  
3   poor_ebp_var = results$MSE$Mean_bench))
```

① `TA_CODE = levels(results$ind$Domain)`: Это идентификатор уровня `admin3`. Он хранится как факторная переменная, поэтому мы извлекаем уровни, а не значения.

³⁹ Результат выполнения кода здесь не показан, но вы можете выполнить его самостоятельно.

- ② `poor_ebp = results$ind$Mean`: Это смоделированный уровень бедности *после задания эталонных значений*. Он хранится в объекте `ind`, который представляет собой смоделированные оценки. Обратите внимание, что объект `ind` также содержит оценки без заданных эталонных значений (в столбце `Mean`).
- ③ `poor_ebp_var = results$MSE$Mean_bench`: Это оценка смоделированной дисперсии. Она хранится в объекте `MSE`, который содержит оценки смоделированной дисперсии. Здесь применимо то же замечание о задании эталонных значений.

Таблица 2: Сравнение выборочных и вневыборочных оценок

	Прямая оценка		Оценка ЕВР	
	Уровень	SE	Уровень	SE
Выборка	0,318	0,128	0,306	0,070
Вне выборки			0,231	0,114

Примечание: В таблице представлены средние уровни бедности и стандартные ошибки (SE) при использовании прямых оценок (столбцы один и два) и оценок наилучшего эмпирического прогноза (ЕВР) (столбцы три и четыре).

Далее проведем оценку выборочных оценок. Это можно сделать с помощью функции `direct()` из пакета `rovmmap`. Для этого мы будем использовать данные на *уровне домохозяйств*, а не данные уровня `admin4`. Необработанные данные опроса хранятся в виде файла Stata, поэтому мы прочитаем их с помощью функции `read_dta()` из пакета `haven`⁴⁰.

```

1 hhs <- read_dta("data/ihshousehold/ihs5_consumption_aggregate.dta")
2 hhs$weights <- hhs$adulseq*hhs$hh_wgt
3 estimates <- direct(
4   y = "poor",
5   smp_data = hhs,
6   smp_domains = "TA",
7   weights = "weights",
8   var = TRUE,
9   HT = TRUE
10  )
11 resultsdirect <- as_tibble(cbind(TA_CODE = levels(estimates$ind$Domain),
12   poor_dir = estimates$ind$Mean,
13   poor_dir_var = estimates$MSE$Mean))

```

Теперь соединим их вместе и переведем значения в числовые величины, а также извлечем квадратный корень из оценок дисперсий, чтобы представить их в виде стандартных ошибок:

⁴⁰Обратите внимание, что мы создаем новую весовую переменную для оценки бедности по численности населения.

```

1 resultsall <- resultsebp |>
2   left_join(resultsdirect, by = "TA_CODE")
3 resultsall <- resultsall |>
4   mutate(poor_ebp = as.numeric(poor_ebp),
5          poor_ebp_var = sqrt(as.numeric(poor_ebp_var)),
6          poor_dir = as.numeric(poor_dir),
7          poor_dir_var = sqrt(as.numeric(poor_dir_var)))

```

И вот наши результаты:

Средние показатели бедности весьма схожи, но обратите внимание на то, что стандартные ошибки у смоделированных оценок гораздо меньше. Это объясняется тем, что модель может «заимствовать предиктивную способность» из других областей.

В этом примере средняя стандартная ошибка уменьшается примерно на 45 процентов, что эквивалентно повышению точности за счет увеличения размера обследования примерно в 3,3 раза.

7.4.3 Как мы можем проверить точность?

Проверка точности результатов может быть очень сложной. При разработке новых методов мы часто используем данные переписи населения для получения информации о «реальной ситуации», которую мы можем сравнить с нашими смоделированными оценками. Конечно, если бы у нас всегда был доступ к оценкам бедности, полученным в ходе переписи населения, нам бы вообще не пришлось оценивать модель малых районов.

В нашем же случае, при наличии выборки мы можем использовать сами данные выборки для проверки точности. Каким образом? С помощью перекрестной проверки. Мы можем оценить модель на подмножестве данных, а затем спрогнозировать ее на оставшихся данных, подобно тому, как мы делали это выше с помощью метода лассо-регрессии для отбора переменных. Вот очень простой пример, в котором мы оцениваем бедность только для одной «свертки», или одной пятой части данных. При этом, важно понимать, что мы оцениваем уровень бедности на географическом уровне admin3 (ТВ), а это значит, что нам нужно присвоить уровень admin4 (СУ) сверткам на уровне admin3.

```

1 # получим данные ТВ
2 povTAs <- unique(pov$TA_CODE)
3 # определим свертки - зададим начальное значение для последовательности полученных
  ↳ результатов
4 set.seed(2025)
5 povTAs <- cbind(TA_CODE = povTAs, foldTA = sample(1:5, length(povTAs), replace =
  ↳ TRUE))
6 # добавим назад в объект pov
7 pov <- pov |>
8   left_join(as_tibble(povTAs), by = "TA_CODE")
9
10 resultscv <- ebp(
11   fixed = ebpformula,
12   pop_data = features,

```

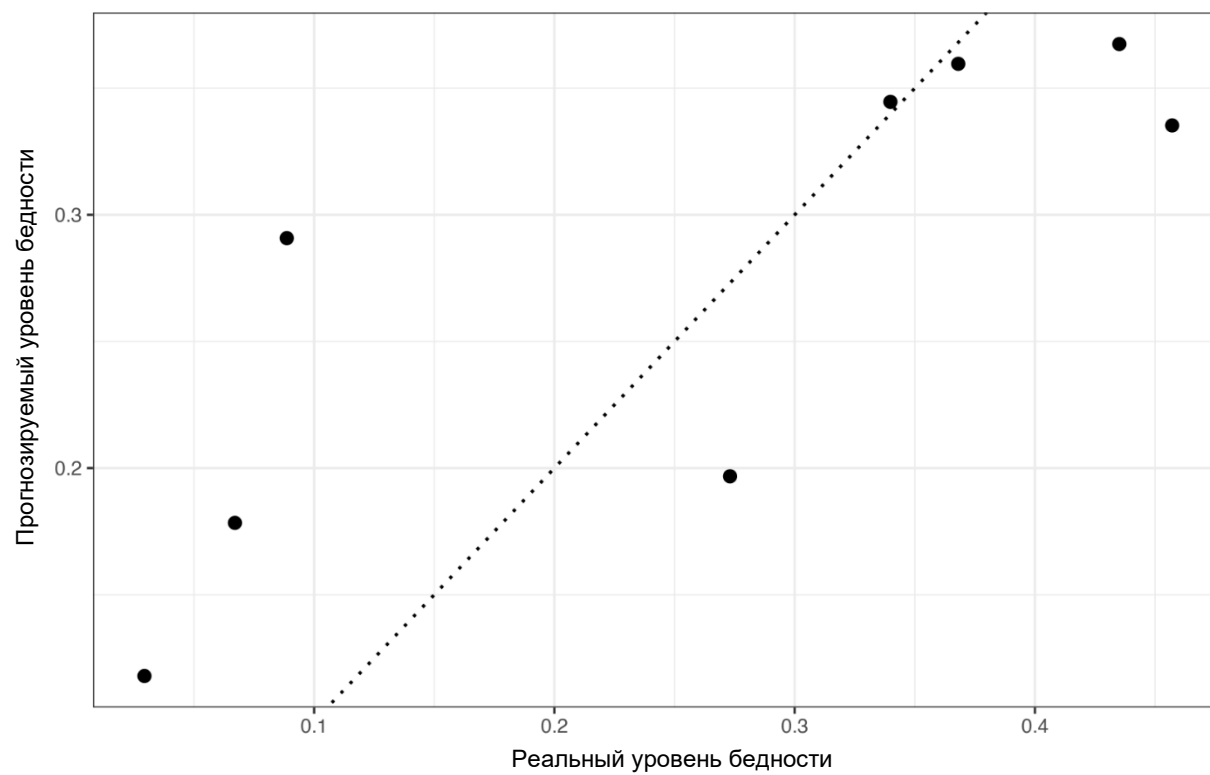
```

13 pop_domains = "TA_CODE",
14 smp_data = pov[pov$foldTA!=1,],
15 smp_domains = "TA_CODE",
16 MSE = TRUE,
17 transformation = "arcsin",
18 na.rm = TRUE,
19 weights = "total_weights",
20 pop_weights = "mwpop",
21 weights_type = "nlme",
22 benchmark = bench
23 )
24 povfold1 <- pov |>
25   filter(foldTA==1) |>
26   group_by(TA_CODE) |>
27   summarise(poor = weighted.mean(poor, total_weights, na.rm = TRUE)) |>
28   ungroup()
29 resultscv <- as_tibble(cbind(TA_CODE = levels(resultscv$ind$Domain),
30   poor_ebp = resultscv$ind$Mean_bench,
31   poor_ebp_var = resultscv$MSE$Mean_bench))
32 # переведем в числа, чтобы объединить с povfold1
33 resultscv$TA_CODE <- as.numeric(resultscv$TA_CODE)
34 resultscv <- povfold1 |>
35   left_join(resultscv, by = "TA_CODE") |>
36   mutate(poor_ebp = as.numeric(poor_ebp),
37   poor_ebp_var = sqrt(as.numeric(poor_ebp_var)))
38 resultscv

```

Теперь мы можем посмотреть на статистику, например, на корреляции между смоделированными и выборочными оценками, а также среднеквадратичную ошибку. Мы также можем визуализировать результаты, как показано на рис. 16.

Рисунок 16: Прогнозируемый и реальный уровень бедности



8 Представление бедности в виде карты распределения

Завершающим шагом будет отображение результатов на карте. Мы можем сделать это, соединив расчетные уровни бедности с файлом формы уровня admin3. Давайте сначала загрузим файл формы, а затем соединим результаты:

```
1 admin3 <- vect("data/mw3.shp")
2 admin3 <- admin3 |>
3   left_join(resultsall, by = "TA_CODE")
4 admin3
```



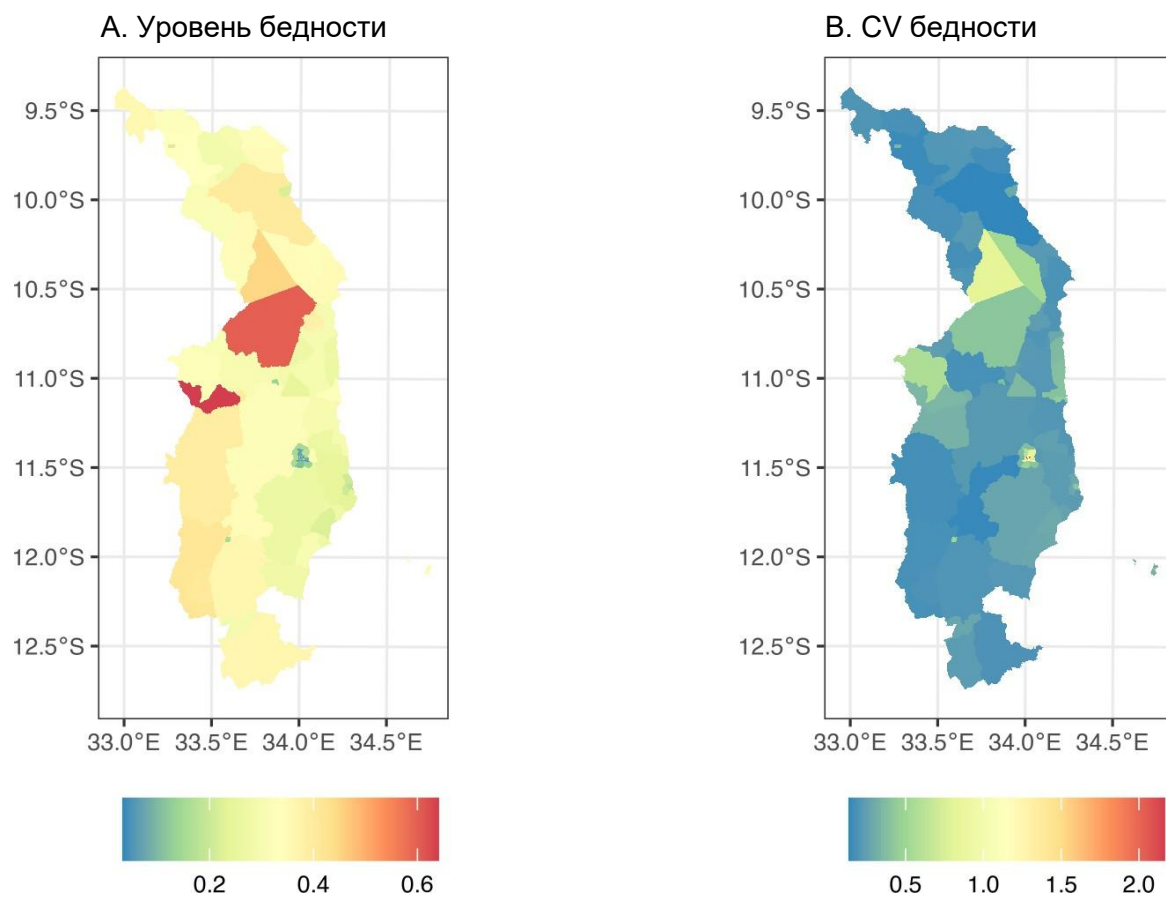
```
class      : SpatVector
geometry    : polygons
dimensions  : 76, 6 (geometries, attributes)
extent      : 493675.9, 691460, 8591761, 8964834 (xmin, xmax, ymin, ymax)
coord. ref. : Arc 1950 / UTM zone 36S
names       : TA_CODE  DIST_CODE  poor_ebp  poor_ebp_var  poor_dir  poor_dir_var
type        : <chr>    <chr>      <num>    <num>      <num>    <num>
values      : 10120    101      0.2175   0.07543    0.3177   0.1326
              10110    101      0.4475   0.3654     NA       NA
              10102    101      0.3263   0.05838    0.3463   0.08992
```

Наконец, мы можем отобразить на карте уровни бедности и стандартные ошибки. Более конкретно, отобразим уровень бедности и коэффициент вариации, который определяется как стандартная ошибка, деленная на среднее значение. Вот код:

```
1 g1 <- ggplot() +
2   geom_spatvector(data = admin3, aes(fill = poor_ebp), color = NA) +
3   scale_fill_distiller("", palette = "Spectral") +
4   labs(subtitle = "A. Poverty rate") +
5   theme_bw()
6 g2 <- ggplot() +
7   geom_spatvector(data = admin3, aes(fill = poor_ebp_var/poor_ebp), color = NA) +
8   scale_fill_distiller("", palette = "Spectral") +
9   labs(subtitle = "B. Poverty CV")
10  + theme_bw()
```

Результат представлен на рис. 17. Следует отметить, что коэффициент вариации (CV) может вводить в заблуждение в отношении районов с очень низким уровнем бедности. Это особенно верно для столицы Северной провинции Малави, где уровень бедности ниже 0,1, что приводит к одним из самых высоких показателей CV для этой провинции.

Рисунок 17: Карты бедности в Северной провинции Малави



9 Подведение итогов

В этом руководстве приведен практический пример применения методов оценки малых районов (ОМР) с использованием геопространственных данных с особым акцентом на оценку бедности на уровне admin3 в Северной провинции Малави. Мы рассмотрели каждый этап процесса – от настройки среды языка программирования R и работы с векторными и растровыми данными до подготовки наборов данных обследования, построения пространственных характеристик, оценки моделей ОМР и отображения конечных результатов в виде карты. Несмотря на то, что пример, используемый здесь, относится к Малави, общий подход рассчитан на универсальность. При наличии подходящих данных те же методы можно применять в условиях самых разных стран и для различных показателей, связанных с ЦУР.

Следует отметить, что мы не утверждаем, что использованные здесь методы и данные всегда должны быть отправной точкой для практикующего специалиста по ОМР. Например, если имеются свежие данные переписи населения, мы бы рекомендовали практикам вместо рассмотренных здесь моделей по субрайонам использовать методы, подходящие для данных переписи, т.е., например, модели на уровне домохозяйств. Важно изучить все возможные доступные данные и выбрать методы, подходящие для этих данных.

Мы также рекомендуем пользователям ознакомиться с другими доступными ресурсами, включая [электронную энциклопедию проекта SAE4SDGs](#) и [начальный учебник по оценке малых территорий с использованием геопространственных данных](#).

Библиографический указатель

- Ghosh, Malay. 2020. "Small area estimation: Its evolution in five decades." *Statistics in Transition. New Series* 21 (4): 1–22.
- Gorelick, Noel, Matt Hancher, Mike Dixon, Simon Ilyushchenko, David Thau, and Rebecca Moore. 2017. "Google Earth Engine: Planetary-Scale Geospatial Analysis for Everyone." *Remote Sensing of Environment*. <https://doi.org/10.1016/j.rse.2017.06.031>.
- Molina, Isabel, and Jon NK Rao. 2010. "Small Area Estimation of Poverty Indicators." *Canadian Journal of Statistics* 38 (3): 369–85.
- Rao, John NK, and Isabel Molina. 2015. *Small Area Estimation*. John Wiley & Sons.
- Rolf, Esther, Jonathan Proctor, Tamma Carleton, Ian Bolliger, Vaishaal Shankar, Miyabi Ishihara, Benjamin Recht, and Solomon Hsiang. 2021. "A Generalizable and Accessible Approach to Machine Learning with Global Satellite Imagery." *Nature Communications* 12 (1): 4392.

Приложение

Рисунок А1: Ограничительная рамка карты Северной провинции Малави

